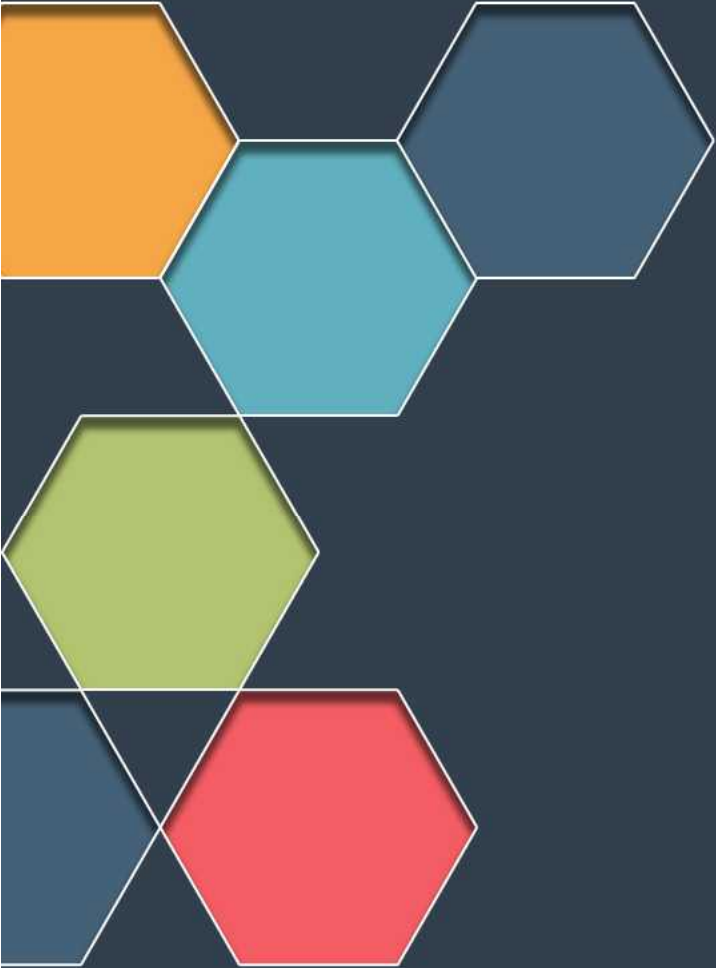


A decorative graphic on the left side of the slide consists of six overlapping hexagons. The colors of the hexagons are orange, teal, dark blue, light green, dark blue, and red, arranged in a staggered, overlapping pattern.

제13장 신경망II (MLP)

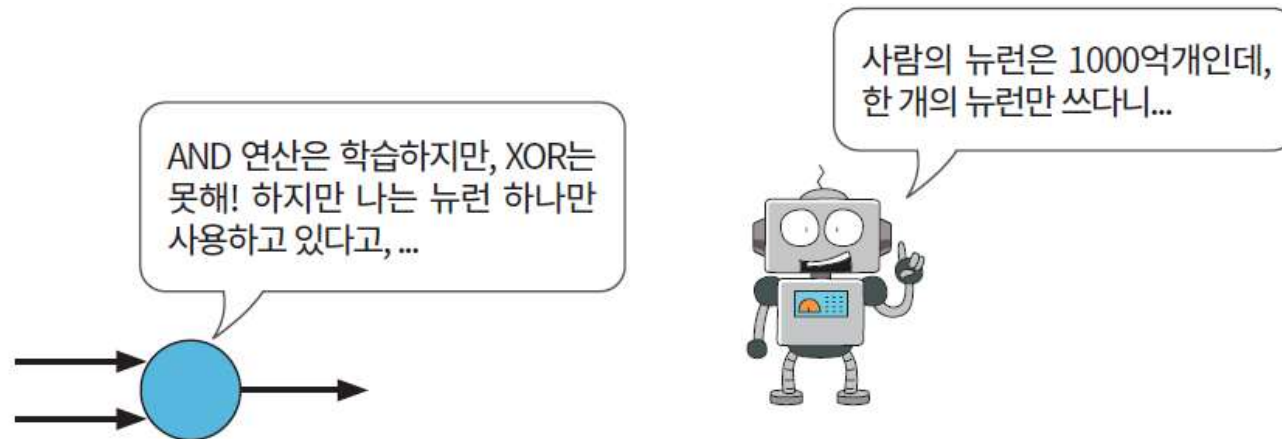
학습 목표

- MLP의 작동 원리를 이해한다.
- MLP의 학습 알고리즘(역전파 알고리즘)을 이해한다.
- Keras 라이브러리로 MLP를 구현해본다.



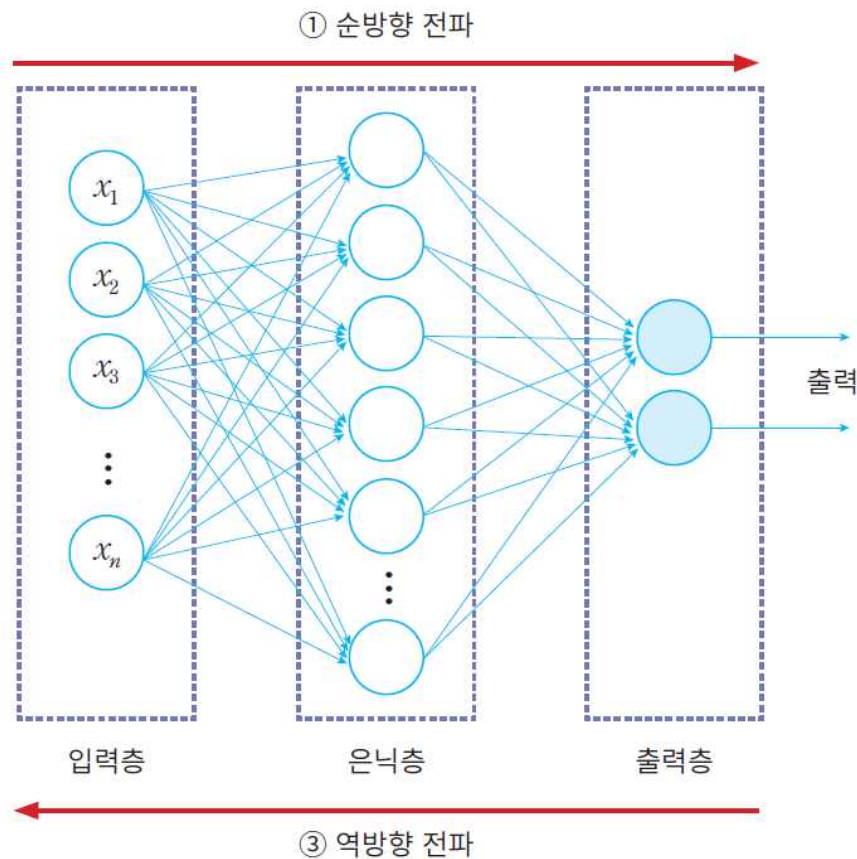
퍼셉트론의 문제점

- 퍼셉트론은 XOR 연산도 올바르게 학습할 수 없었다.
- 퍼셉트론의 구조가 너무 간단하였다는 데 있다. 인간의 두뇌에는 약 860억개 뉴런들이 서로 협동하여 복잡한 인지 문제들을 해결한다. 하지만 퍼셉트론에서는 단 하나의 뉴런 만을 사용하고 있다.



01 MLP

- 다층 퍼셉트론(multilayer perceptron: MLP): 입력층과 출력층 사이에 은닉층(hidden layer)을 가지고 있는 신경망



MLP는 뉴런의 개수를 대폭 늘리고, 은닉층을 도입하였습니다. 하지만 어떻게 학습시키나요?

연결마다 있는 가중치의 학습 방법을 알게되었다.

② 오차 계산

역전파 학습 알고리즘

- 역전파 알고리즘은 입력이 주어지면 순방향으로 계산하여 출력을 계산한 후에 실제 출력과 우리가 원하는 출력 간의 오차를 계산한다.
- 이 오차를 역방향으로 전파하면서 오차를 줄이는 방향으로 가중치를 변경한다.

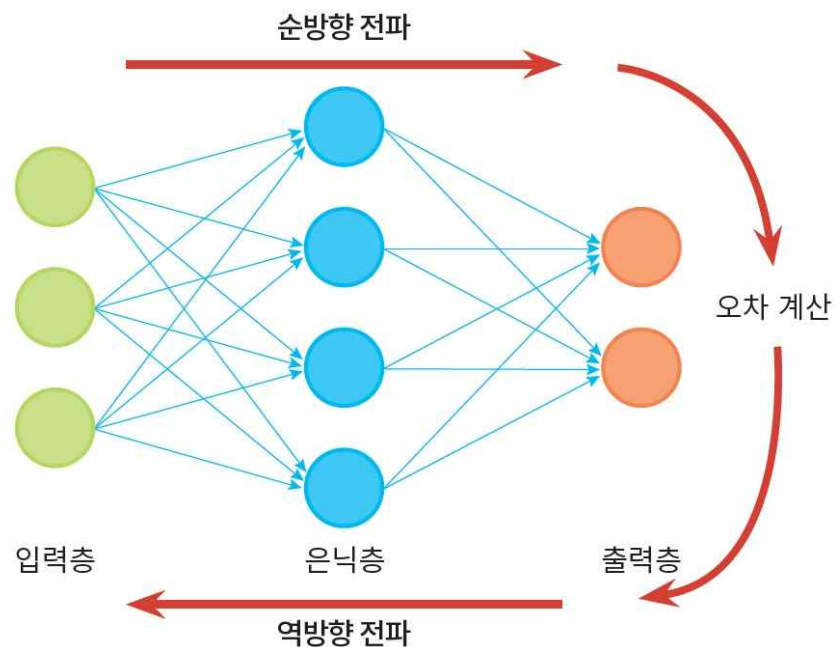


그림 13-2 역전파 학습 알고리즘

활성화 함수

- 활성화 함수(activation function)은 입력의 총합을 받아서 출력값을 계산하는 함수이다.
- 퍼셉트론에서는 계단 함수(step function)를 활성화 함수로 사용하였지만, **MLP**에서는 다양한 비선형 함수들을 활성화 함수로 사용한다

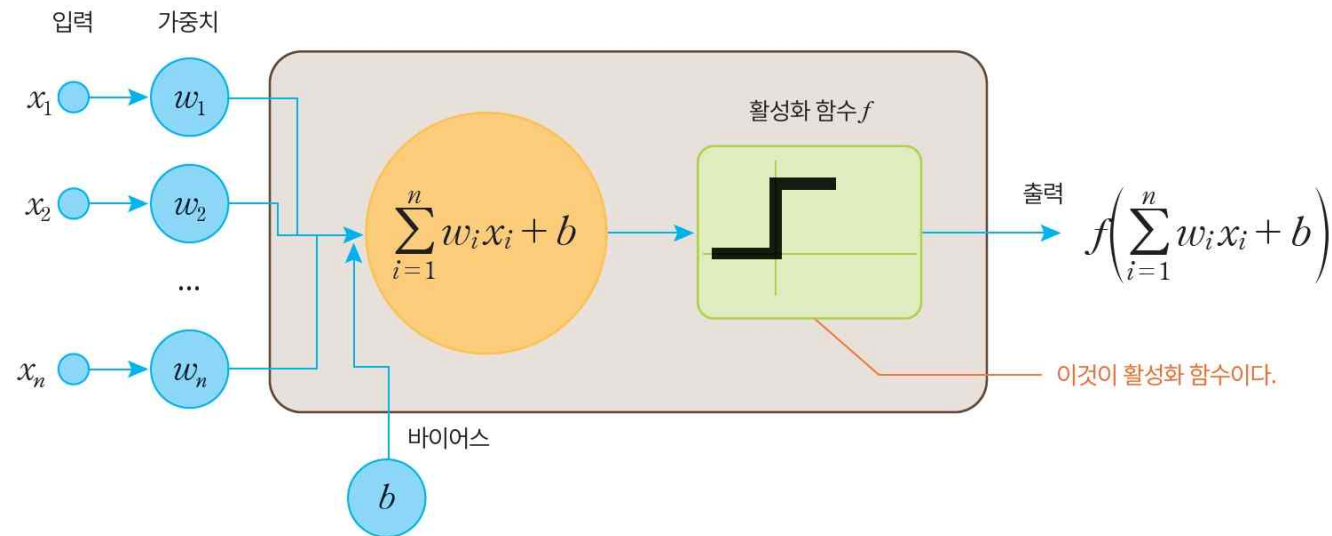
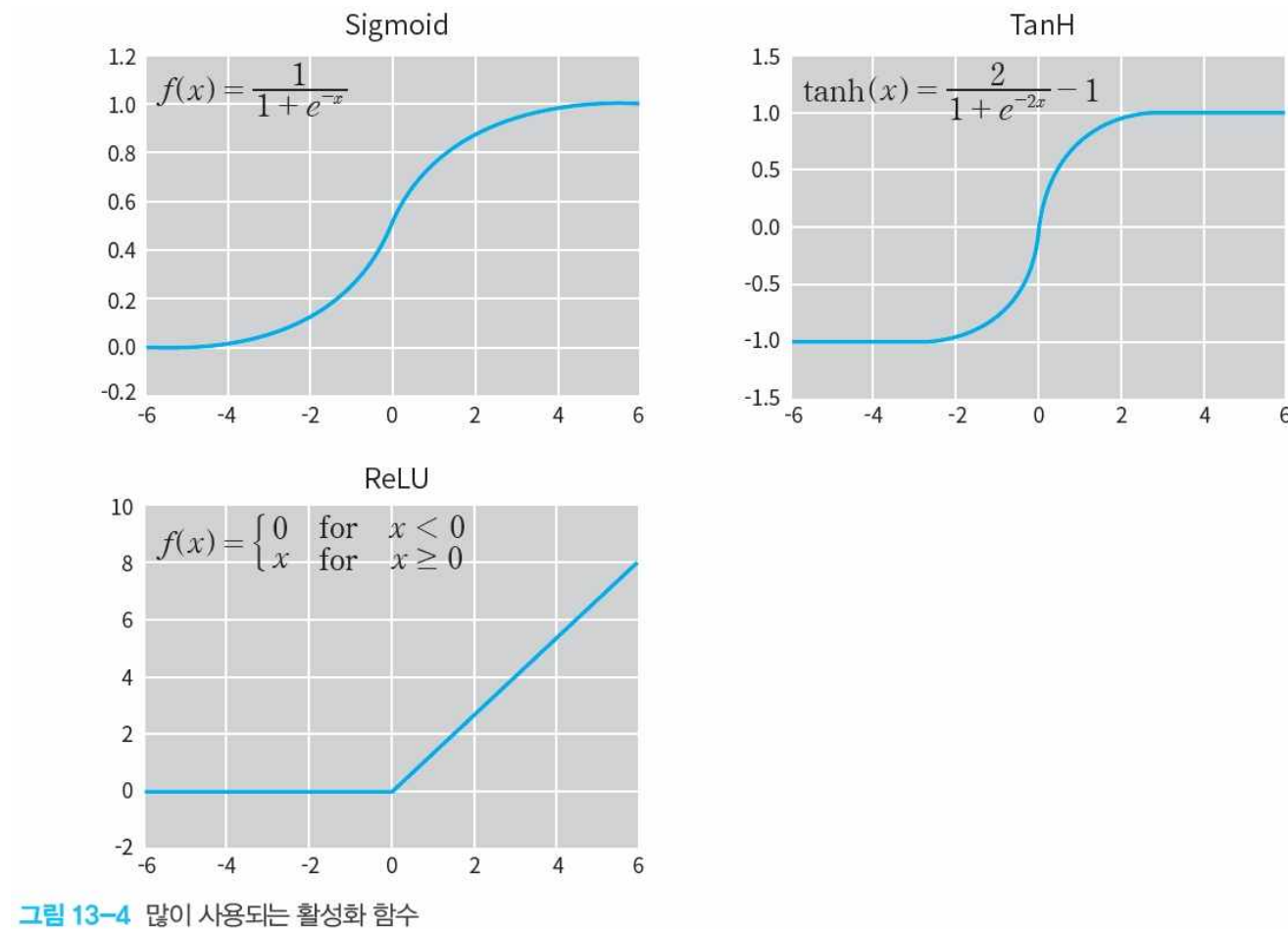


그림 13-3 활성화 함수

활성화 함수



선형 함수는 많이 써도 효과가 없음

- 여러 개의 선형 함수를 결합해도 결국은 선형 함수 하나와 같다는 것이 수학적으로 증명되어 있기 때문이다.

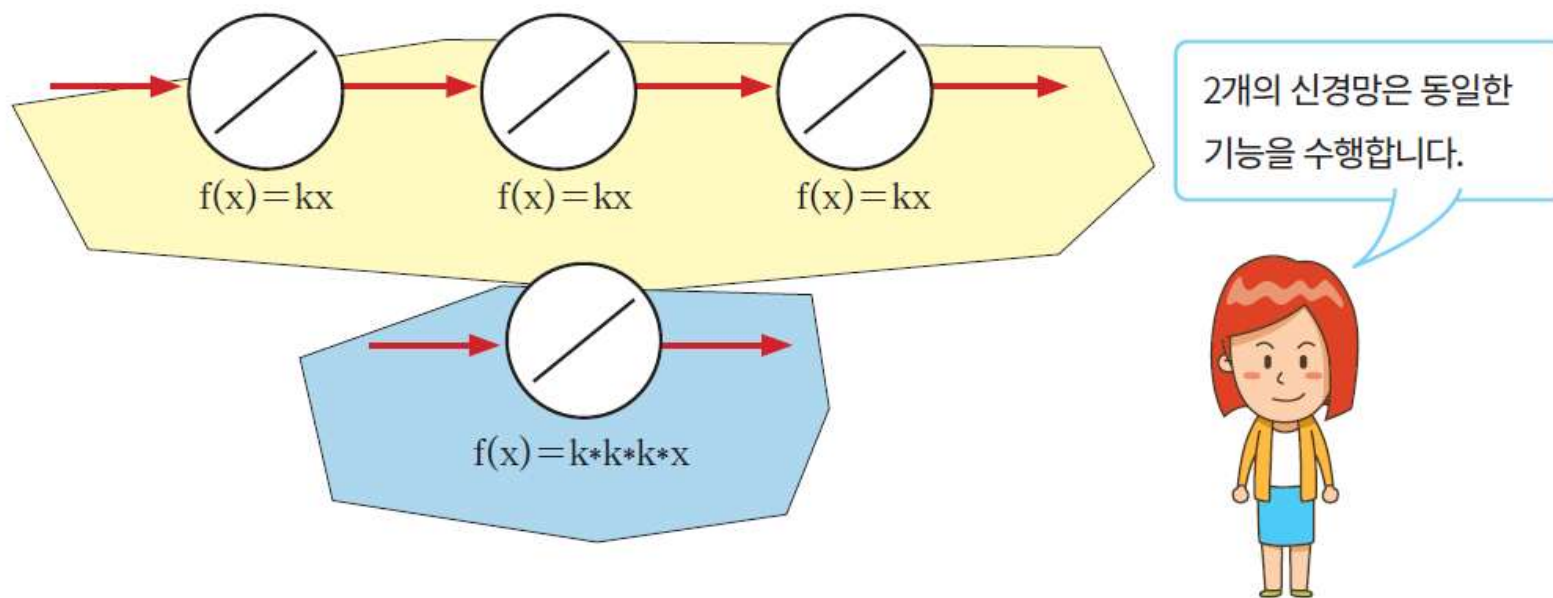


그림 13-4 선형 레이어는 아무리 많아도 하나의 레이어로 대체될 수 있다.

계단 함수(step function)

- 계단 함수는 입력 신호의 총합이 0을 넘으면 1을 출력하고, 그렇지 않으면 0을 출력하는 함수이다.

$$f(x)=\begin{cases} 1 & x>0 \\ 0 & x\leq 0 \end{cases}$$

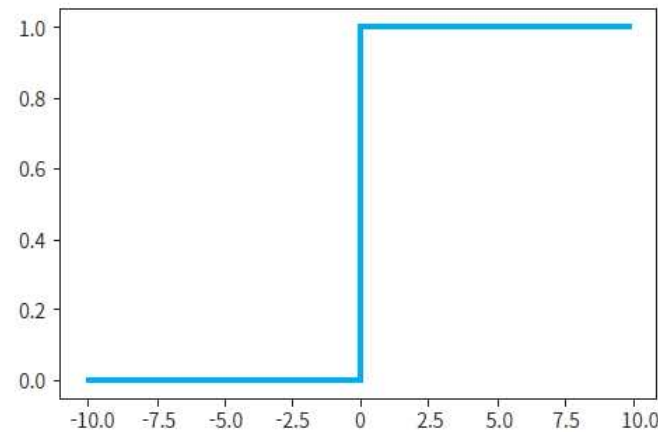


그림 13-5 계단 함수

시그모이드 함수(sigmoid function)

- 1980년대부터 사용되어 온 전통적인 활성화 함수이다. 시그모이드는 다음과 같이 S자와 같은 형태를 가진다.

$$f(x) = \frac{1}{1 + e^{-x}}$$

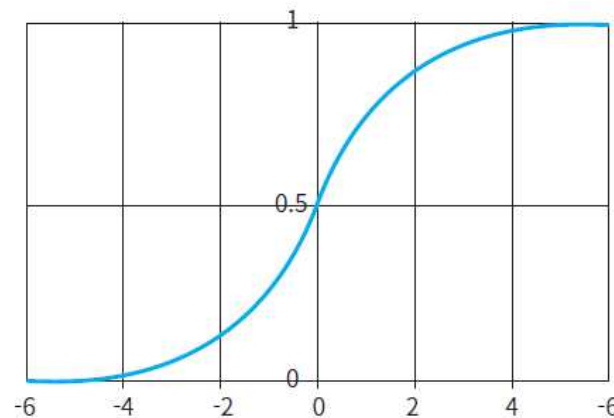


그림 13-6 시그모이드 함수

ReLU 함수(Rectified Linear Unit function)

- ReLU 함수는 입력이 0을 넘으면 그대로 출력하고, 입력이 0보다 적으면 출력은 0이 된다.

$$f(x)=\begin{cases} x & x>0 \\ 0 & x\leq 0 \end{cases}$$

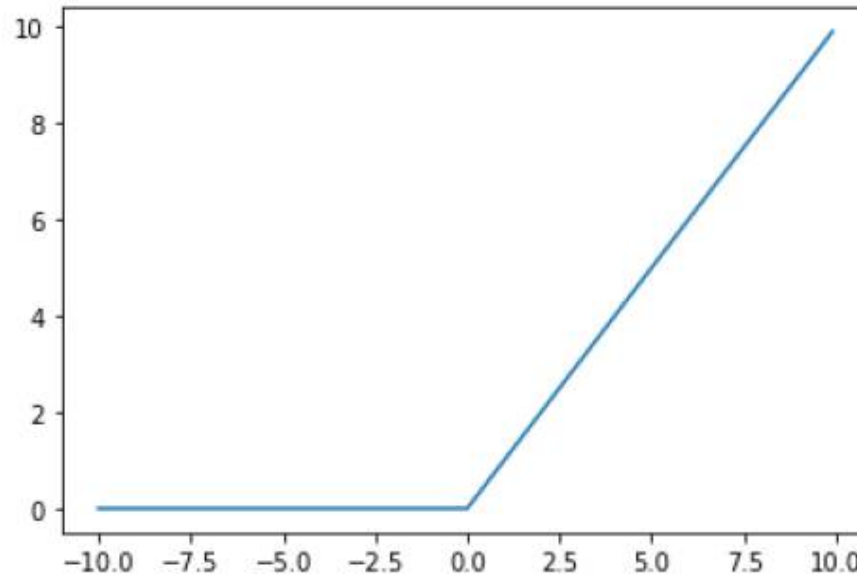


그림 13-7 ReLU 함수

하이퍼볼릭 탄젠트 함수(tanh function)

- $\tanh()$ 함수는 넘파이에서 제공하고 있다. 출력이 -1 에서 $+1$ 까지라는 점이 특이하다.

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{2}{1 + e^{-2x}} - 1$$

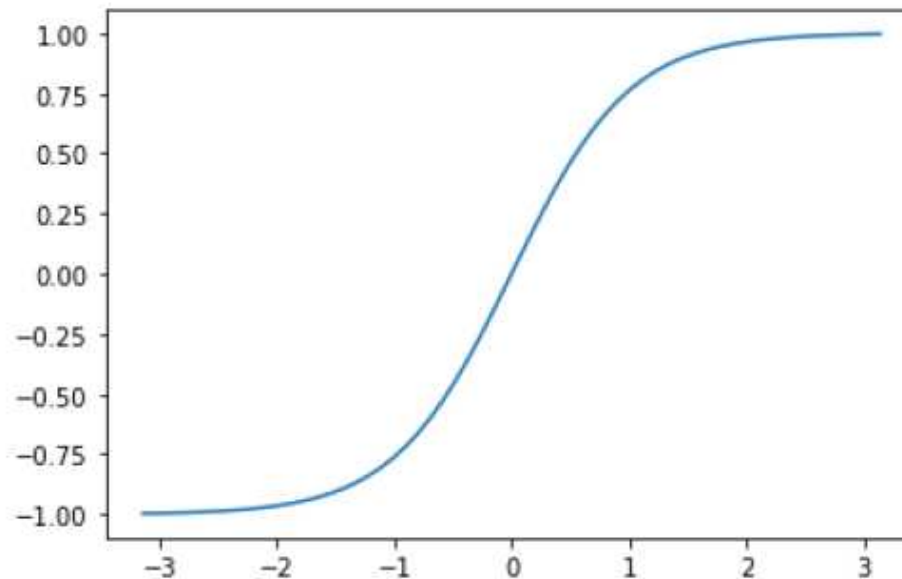
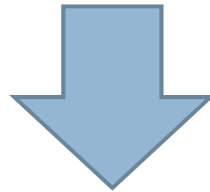


그림 13-8 하이퍼볼릭 탄젠트 함수

Lab: 파이썬으로 활성화 함수 구현하기

```
def step(x):  
    if x > 0.000001: return 1  # 부동 소수점 오차 방지  
    else:            return 0
```



넘파이 배열을 받기 위하여 변경한다.

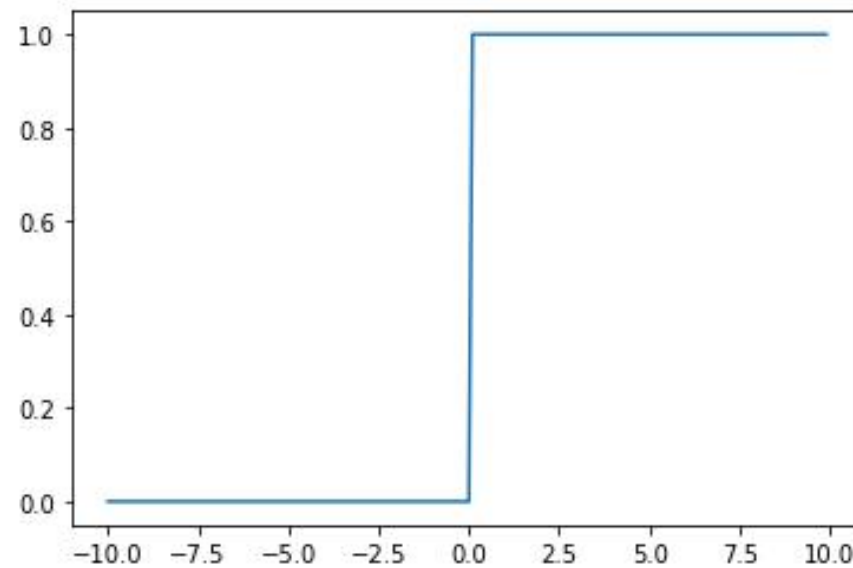
```
def step(x):  
    result = x > 0.000001          # True 또는 False  
    return result.astype(int)      # 정수로 반환
```

x 가 NumPy ndarray 일 때 한 번에 전체 원소를 벡터화(vectorized) 처리하기 위해서 `astype(int)` 를 씁니다. 단순 스칼라 x 라면 `int(result)` 로 충분하지만, 배열 전체에 동시에 적용하려면 `astype` 가 필요합니다.

그래프 그리기

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(-10.0, 10.0, 0.1)
y = step(x)
plt.plot(x, y)
plt.show()
```

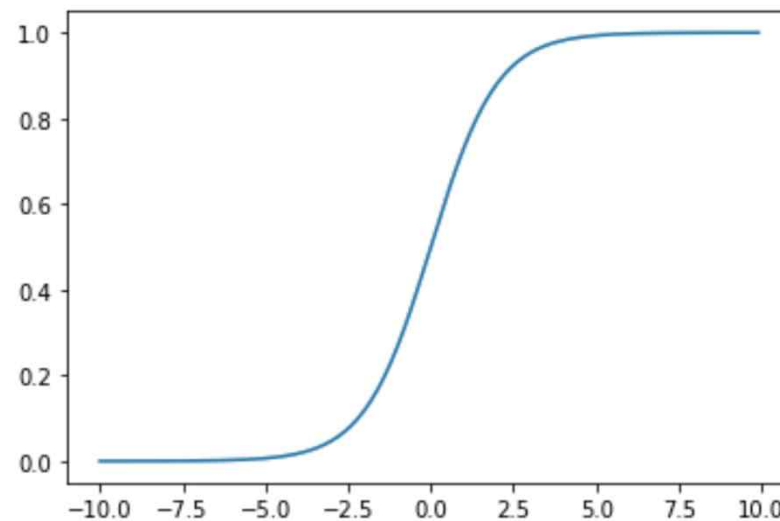


시그모이드 함수

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

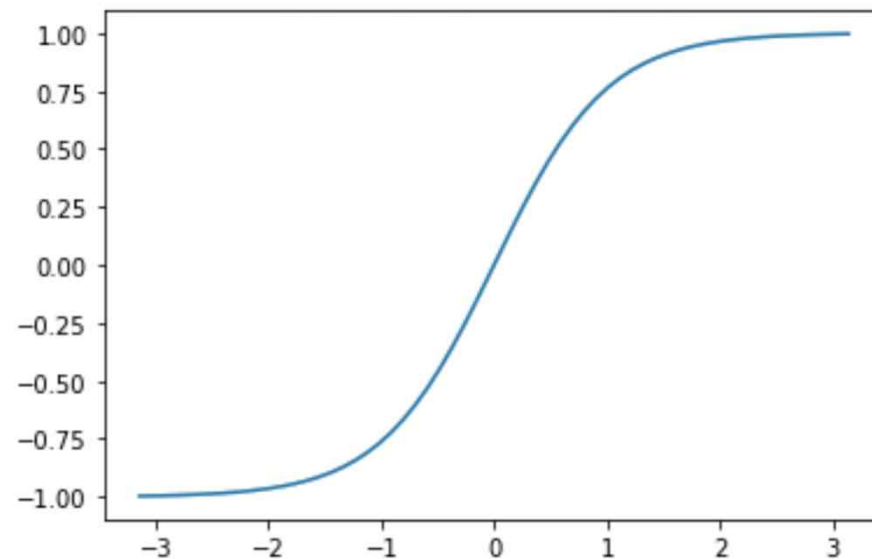
x = np.arange(-10.0, 10.0, 0.1)
y = sigmoid(x)
plt.plot(x, y)
plt.show()
```



tanh 함수

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-np.pi, np.pi, 60)
y = np.tanh(x)
plt.plot(x, y)
plt.show()
```

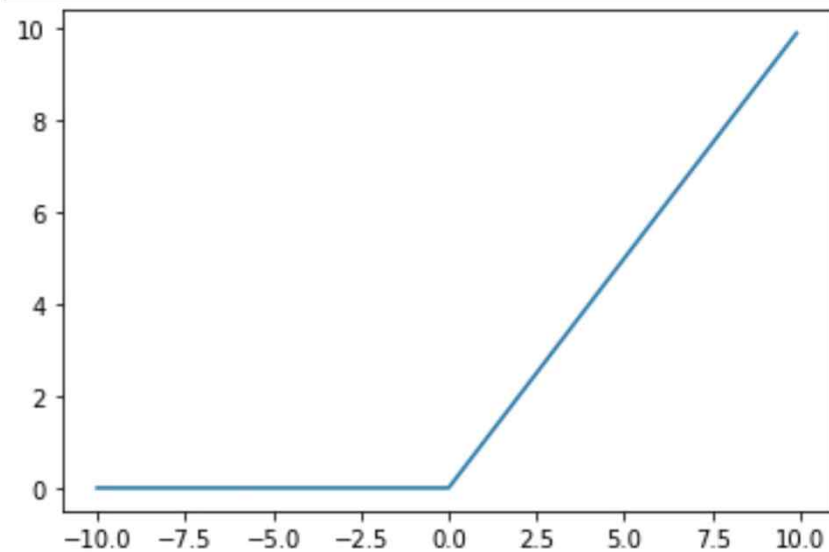


ReLU 함수

```
import numpy as np
import matplotlib.pyplot as plt

def relu(x):
    return np.maximum(x, 0)

x = np.arange(-10.0, 10.0, 0.1)
y = relu(x)
plt.plot(x, y)
plt.show()
```



02 순방향 패스

- 순방향 패스란 입력 신호가 입력층 유닛에 가해지고 이들 입력 신호가 은닉층을 통하여 출력층으로 전파되는 과정을 의미한다.

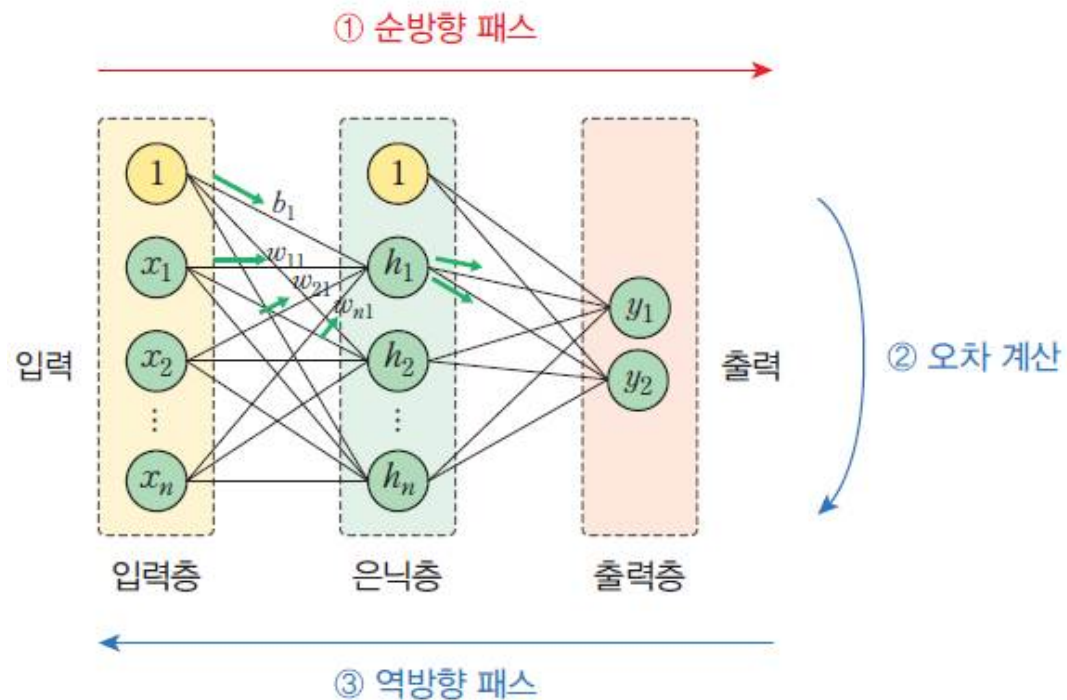
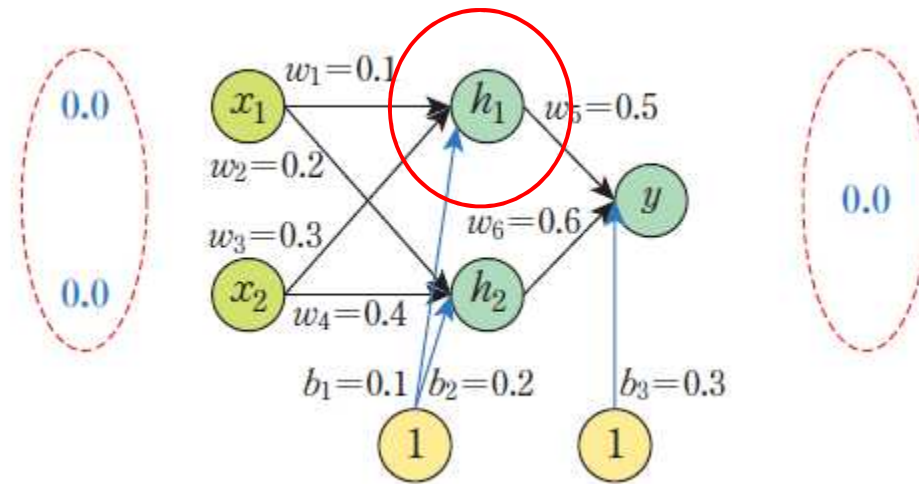


그림 6-5 순방향 패스

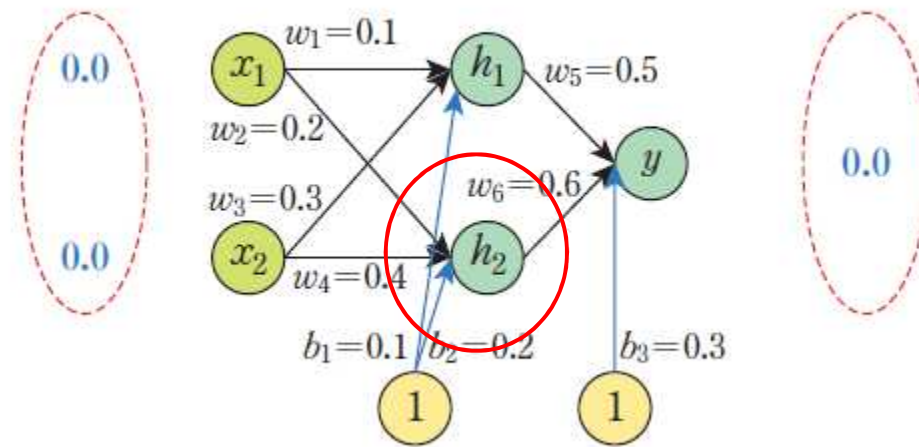
손으로 계산해보자.



$$z_1 = w_1 * x_1 + w_3 * x_2 + b_1 = 0.1 * 0.0 + 0.3 * 0.0 + 0.1 = 0.1$$

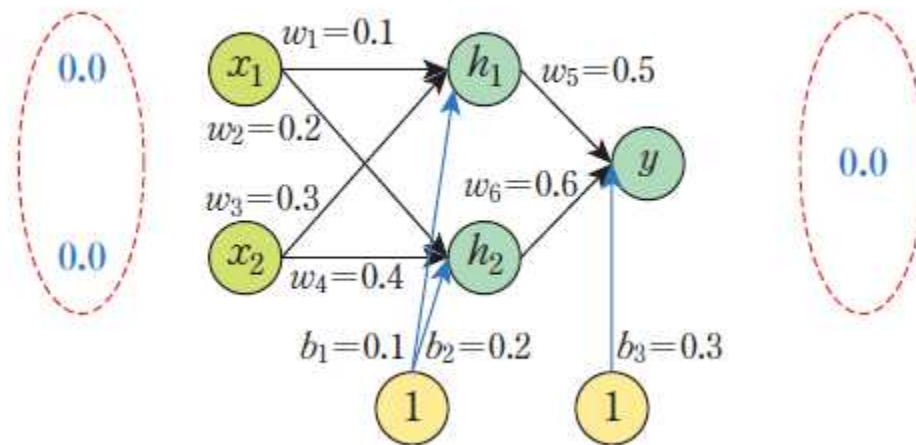
$$a_1 = \frac{1}{1 + e^{-z_1}} = \frac{1}{1 + e^{-0.1}} = 0.524979$$

손으로 계산해보자.



$$a_2 = 0.549834$$

손으로 계산해보자.

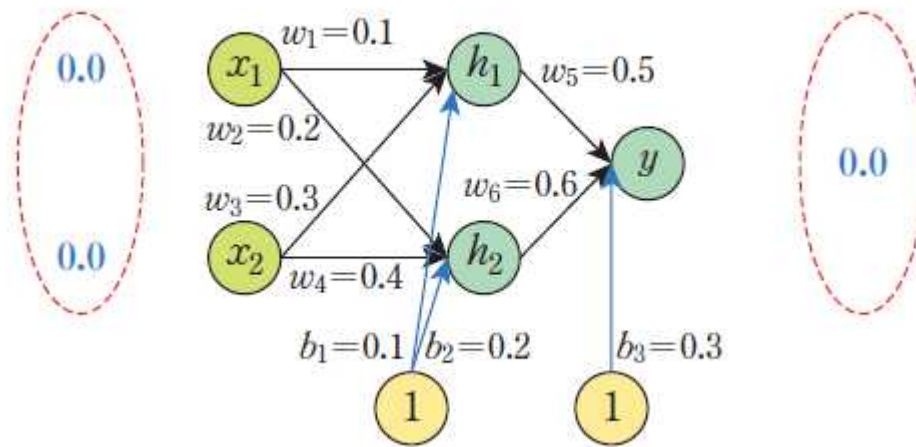


$$\begin{aligned} z_y &= w_5 * a_1 + w_6 * a_2 + b_3 \\ &= 0.5 * 0.524979 + 0.6 * 0.549834 + 0.3 = 0.892389 \end{aligned}$$

$$a_y = \frac{1}{1 + e^{-z_y}} = \frac{1}{1 + e^{-0.892389}} = 0.709383$$

정답은 0이지만 신경망의 출력은 0.71 정도이다. 오차가 상당함을 알 수 있다.

행렬로 표시해보자.



$$z_1 = w_1 * x_1 + w_3 * x_2 + b_1$$

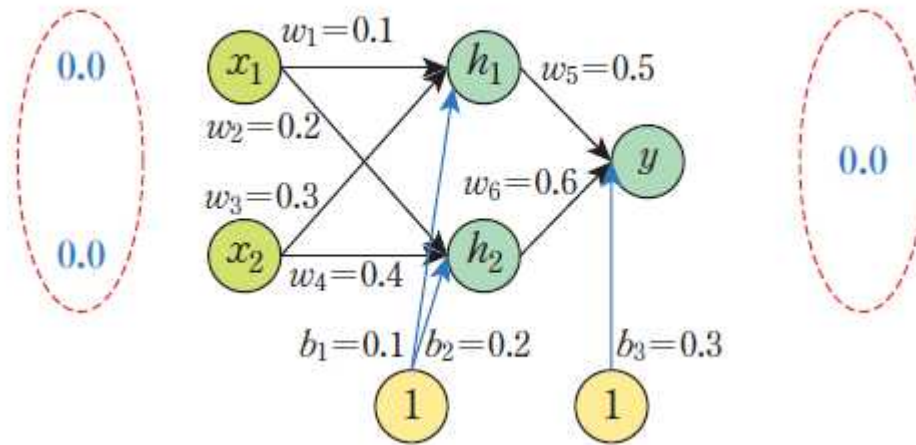
$$z_2 = w_2 * x_1 + w_4 * x_2 + b_2$$



$$Z_1 = XW_1 + B_1$$

행렬로 표시할 수 있다.

행렬로 표시해보자.



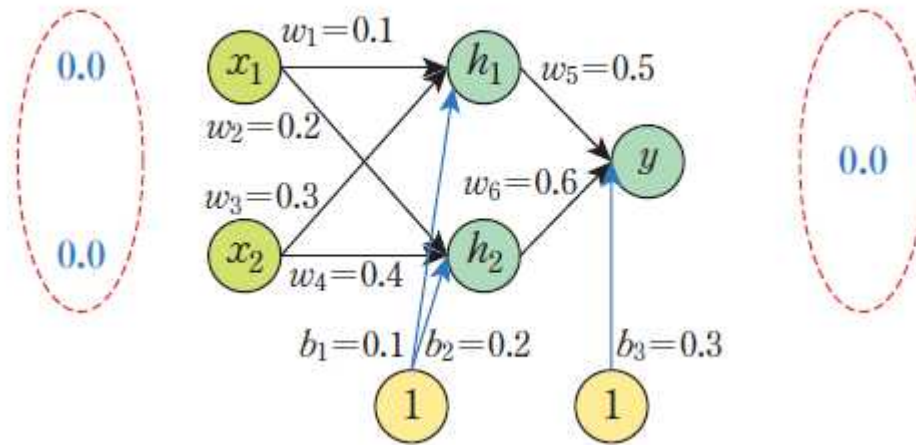
$$X = [x_1 \ x_2], \quad B_1 = [b_1 \ b_2], \quad Z_1 = [z_1 \ z_2]$$

$$W_1 = \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix}$$



$$Z_1 = [z_1 \ z_2] = XW_1 + B_1 = [x_1 \ x_2] \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix} + [b_1 \ b_2]$$

행렬로 표시해보자.



$$W_2 = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix}$$

$$Z_2 = A_1 W_2 + B_2 = \begin{bmatrix} a_1 & a_2 \end{bmatrix} \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} + \begin{bmatrix} b_3 \end{bmatrix}$$

$$A_2 = [y] = f(Z_2)$$

순방향 패스

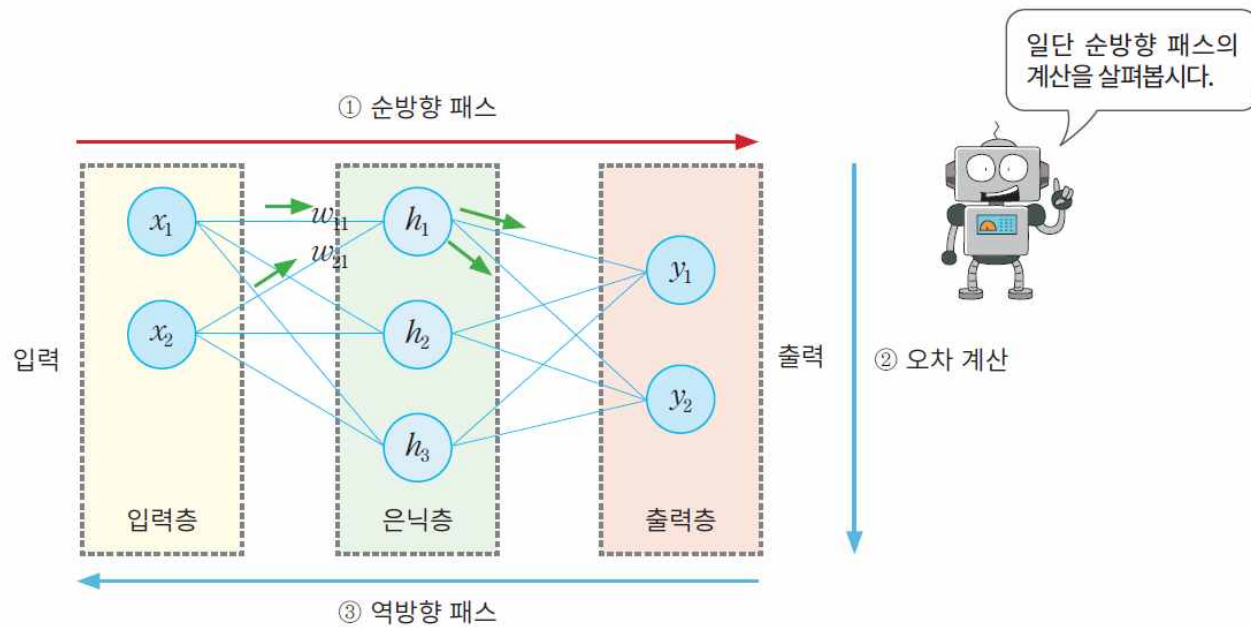


그림 13-9 순방향 패스

$$h_1 = f(w_{11}x_1 + w_{21}x_2 + b_1)$$

$$h_2 = f(w_{12}x_1 + w_{22}x_2 + b_2)$$

$$h_3 = f(w_{13}x_1 + w_{23}x_2 + b_2)$$

행렬의 사용

$$\begin{aligned}h_1 &= f(w_{11}x_1 + w_{21}x_2 + b_1) \\h_2 &= f(w_{12}x_1 + w_{22}x_2 + b_2) \\h_3 &= f(w_{13}x_1 + w_{23}x_2 + b_2)\end{aligned}$$



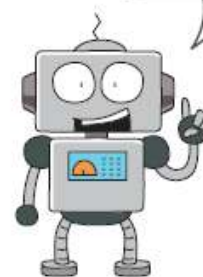
$$[h_1 h_2 h_3] = f\left([x_1 x_2] \cdot \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} + [b_1 b_2 b_3]\right)$$



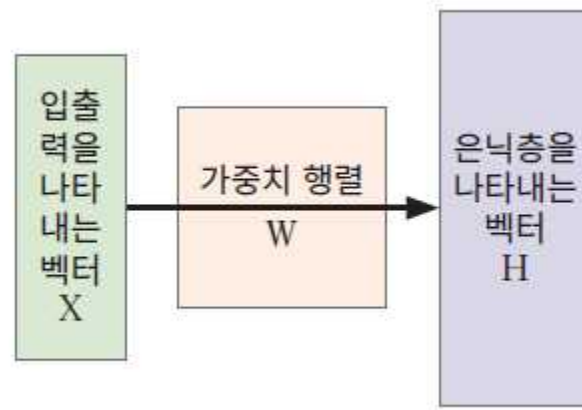
$$H = f(X \cdot W + B)$$

그림 13-10 행렬로 표기된 신경망 연산

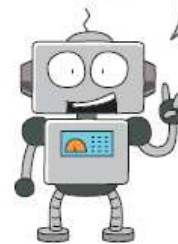
벡터와 행렬의 내적을 풀어쓴
것처럼 보이지 않나요?



행렬의 사용



$$X \times W + B$$



이것은 신경망의 출력을 계산하는 아주 유명한 수식입니다.

Lab: MLP 순방향 패스

```
import numpy as np

# 시그모이드 함수
def actf(x):
    return 1/(1+np.exp(-x))

# 시그모이드 함수의 미분치
def actf_deriv(x):
    return x*(1-x)

# 입력유닛의 개수, 은닉유닛의 개수, 출력유닛의 개수
inputs, hiddens, outputs = 2, 2, 1
learning_rate=0.2

# 훈련 샘플과 정답
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
T = np.array([[0], [1], [1], [0]])
```

Lab: MLP 순방향 패스

```
W1 = np.array([[0.10, 0.20],
                [0.30, 0.40]])
W2 = np.array([[0.50], [0.60]])
B1 = np.array([0.1, 0.2])
B2 = np.array([0.3])

# 순방향 전파 계산
def predict(x):
    layer0 = x
    Z1 = np.dot(layer0, W1)+B1
    layer1 = actf(Z1)
    Z2 = np.dot(layer1, W2)+B2
    layer2 = actf(Z2)
    return layer0, layer1, layer2
```

입력을 **layer0**에 대입한다.
행렬의 곱을 계산한다.
활성화 함수를 적용한다.
행렬의 곱을 계산한다.
활성화 함수를 적용한다.

Lab: MLP 순방향 패스

```
def test():  
    for x, y in zip(X, T):  
        x = np.reshape(x, (1, -1))# x를 2차원 행렬로 만든다.입력은 2차원이어야 한다.  
        layer0, layer1, layer2 = predict(x)  
        print(x, y, layer2)  
test()
```

```
[[0 0]] [0] [[0.70938314]]  
[[0 1]] [1] [[0.72844306]]  
[[1 0]] [1] [[0.71791234]]  
[[1 1]] [0] [[0.73598705]]
```

학습이 없으므로 난
수만 출력된다.

손실 함수 계산

- 신경망에서 학습을 시킬때는 실제 출력과 원하는 출력 사이의 오차를 이용한다. 오차를 계산하는 함수를 손실함수(loss function)라고 한다.

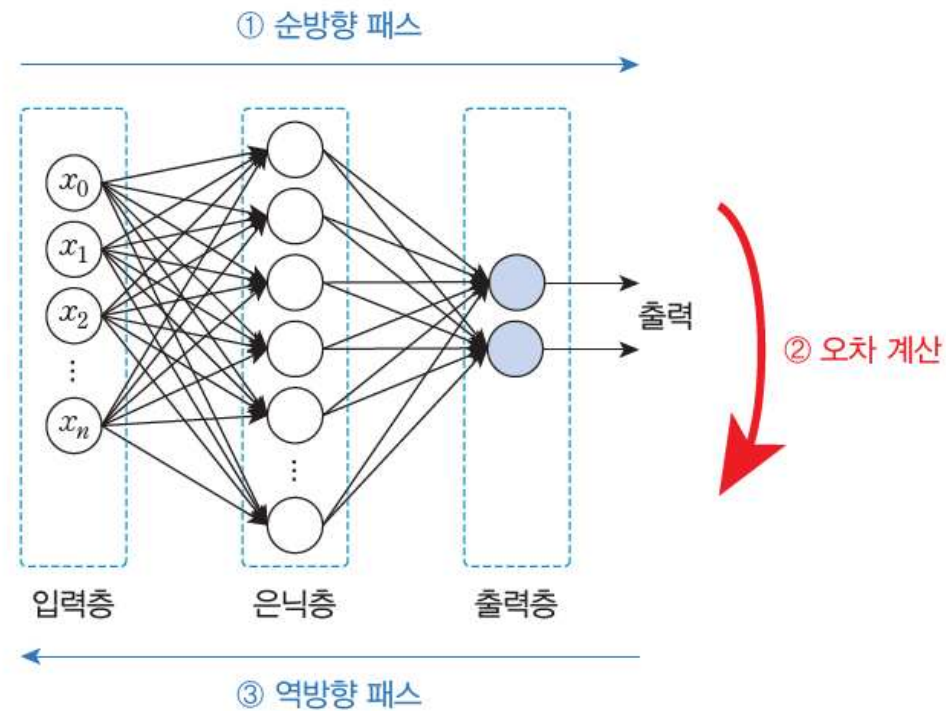
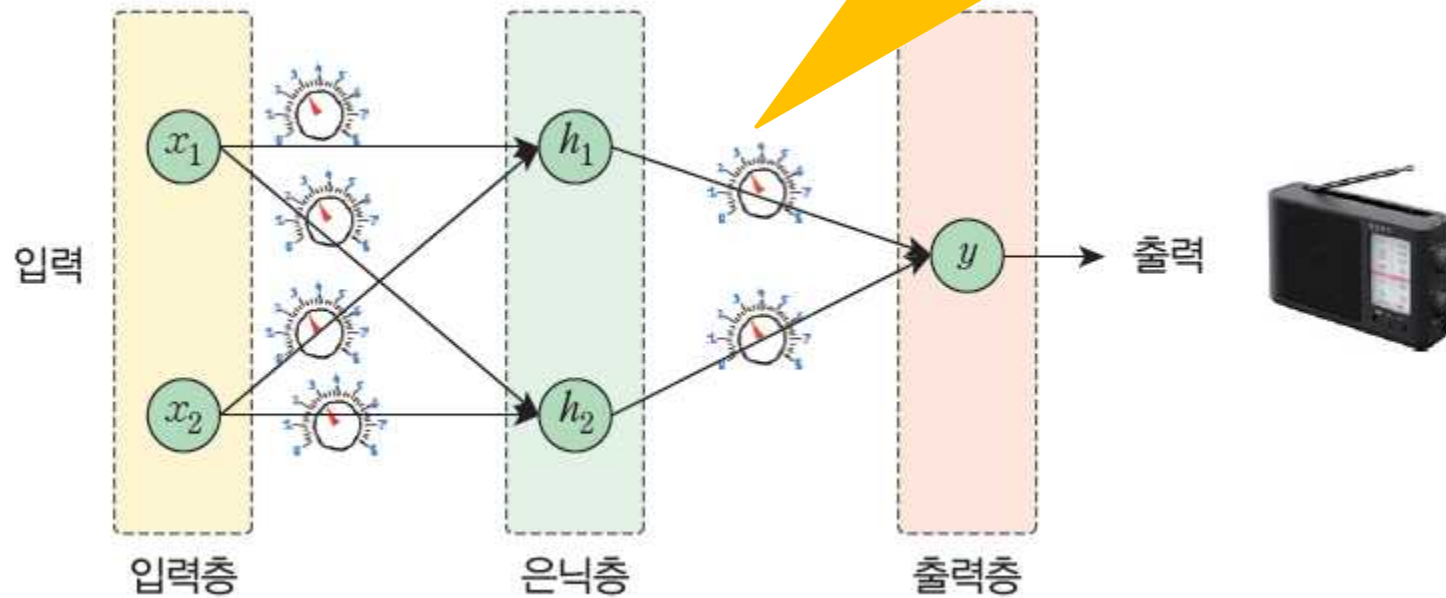


그림 6-6 오차 계산

가중치 == 다이얼

가중치를 조절한다는 것은 스피커에서
나는 소리를 들으면서 튜너 다이얼을 돌
리는 것과 같다.



가중치를 조절하는 과정

- 가중치를 증가시켰을 때 오차가 감소한다. → 가중치를 증가시키면 된다.
- 가중치를 증가시켰을 때 오차가 증가한다. → 가중치를 감소시키면 된다.
- 실제로 가중치를 증가시키거나 감소시켜서 신경망의 출력을 계산하고 이것을 정답과 비교하여 오차를 다시 계산할 수도 있다. 하지만 이 방법은 각 가중치를 변경할 때마다 신경망의 출력을 다시 계산하여야 해서 상당한 시간이 소요된다. 좀 더 좋은 방법은 없을까?

손실 함수

- 신경망에서도 학습의 성과를 나타내는 지표가 있어야 한다. 이것을 손실함수(loss function)이라고 한다

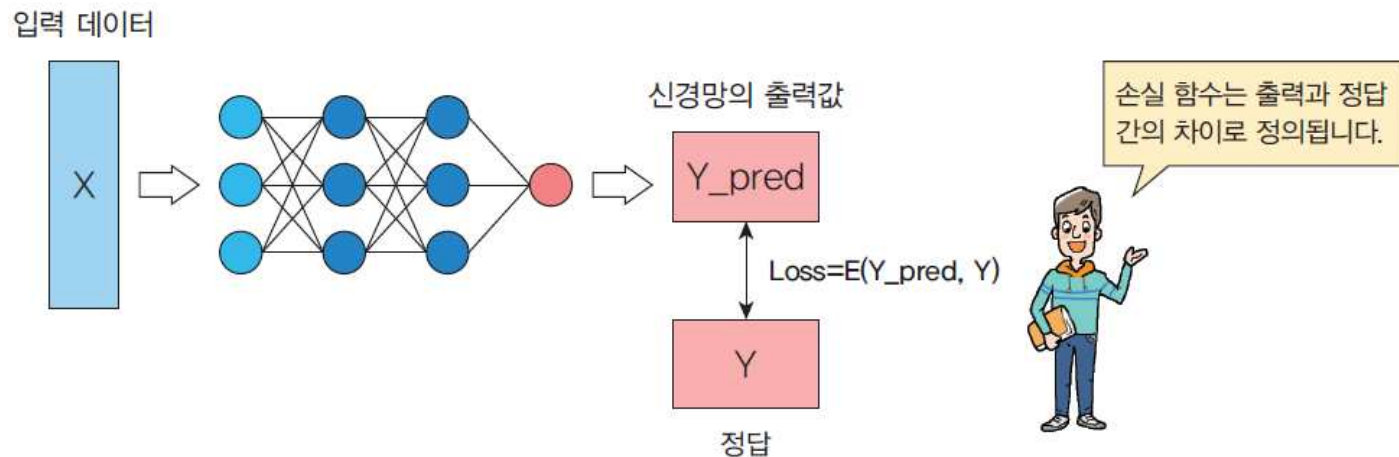


그림 6-7 손실함수의 정의

평균 제곱 오차(MSE)

- 예측값과 정답 간의 평균 제곱 오차

$$E(w) = \frac{1}{2n} \sum_i (y_i - t_i)^2$$

학습 시 미니배치에서는 n 을 생략하고 2로만 나누는 경우가 많지만, 정식 MSE는 반드시 n 으로 나뉘야 합니다.

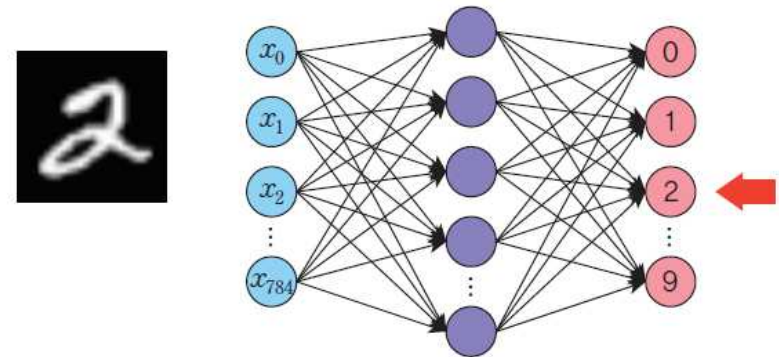


그림 6-8 MNIST 숫자 이미지를 분류하는 신경망

```
>>> y = np.array([ 0.0, 0.0, 0.8, 0.1, 0.0, 0.0, 0.0, 0.1, 0.0, 0.0 ])
>>> target = np.array([ 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 ])

>>> def MSE(target, y):
    return 0.5 * np.sum((y-target)**2)

>>> MSE(target, y)
0.029999999999999992
```

예측값과 정답이 많이 차이는 경우

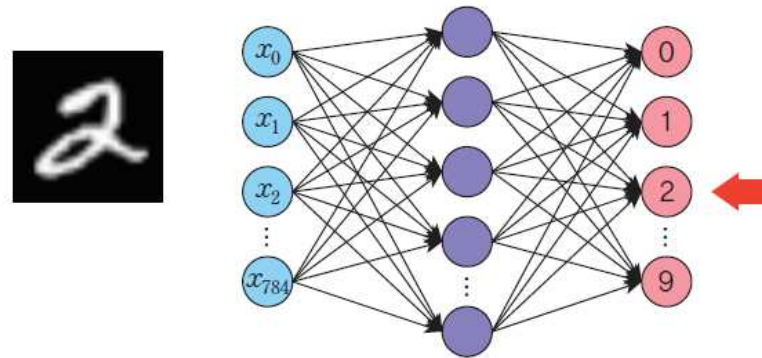


그림 6-8 MNIST 숫자 이미지를 분류하는 신경망

```
>>> y = np.array([ 0.9, 0.0, 0.1, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 ])
>>> target = np.array([ 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 ])

>>> def MSE(target, y):
    return 0.5 * np.sum((y-target)**2)
>>> MSE(target, y)
0.81
```

경사하강법

- 역전파 알고리즘은 신경망 학습 문제를 최적화 문제(optimization)로 접근한다. 우리는 손실함수 값을 최소로 하는 가중치를 찾으려 한다.

$$W^* = \operatorname{argmin}_W E(W)$$

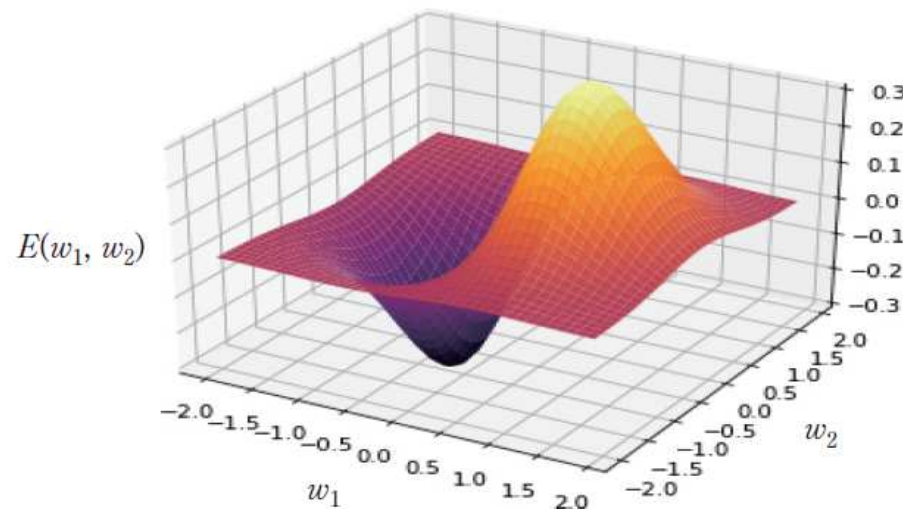


그림 6-9 경사 하강법

그래디언트

- 손실 함수가 신경망처럼 다중 변수 함수일 때, 1차 미분값은 벡터가 되는데 이것을 그래디언트(gradient)라고 한다.

$$\nabla E = \left(\frac{\partial E}{\partial w_1}, \frac{\partial E}{\partial w_2}, \dots, \frac{\partial E}{\partial w_n} \right)$$



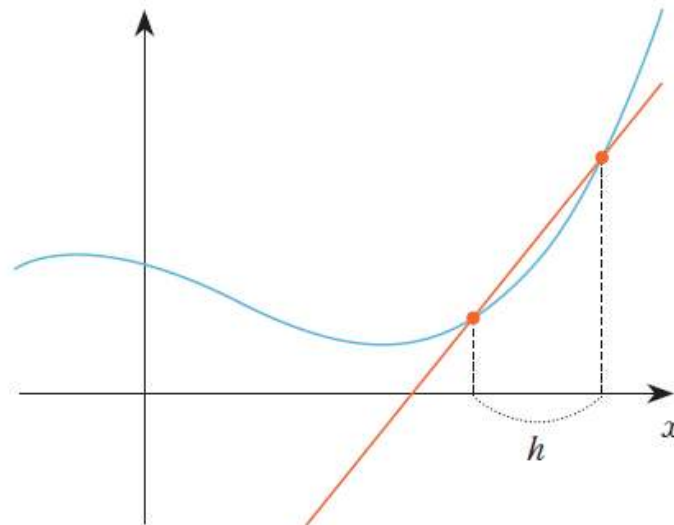
왜 해석적인 방법은 안 되는가?

왜 우리가 중학교 때 학습하였던 손실 함수의 미분값을 0으로 놓아서 최소점을 구하는 방법은 사용되지 않는가? 만약 가중치가 1개 또는 2개만 있다면 손실 함수가 2차 함수가 되어서 가능할 수도 있다. 하지만 실제로 사용되는 신경망에는 수만 개에 이르는 아주 많은 가중치(변수)들이 있을 수 있어서 해석적인 방법은 거의 불가능하다(행렬의 차원이 아주 높아진다. 가중치가 10만개 있다고 상상해 보자). 또 해석적인 방법은, 중간에 값이 하나만 변경되어도 '역행렬 계산'과 같은 작업을 다시 해야 한다. 즉, 점진적 개선이 불가능하다.

경사하강법

- 경사강하법은 함수의 1차 미분값(그래디언트)을 사용하는 반복적인 최적화 알고리즘이다. 미분은 다음과 같이 정의된다.

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$



어떤 점에서 미분값이란 x 를 증가시켰을 때 $f(x)$ 가 얼마나 증가하느냐입니다.

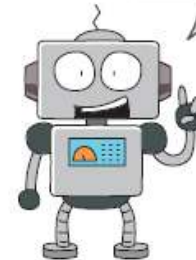
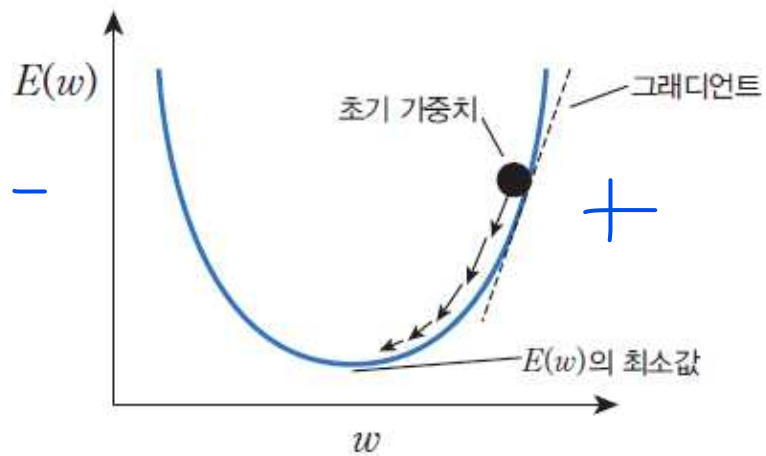


그림 13-18 미분의 정의

경사하강법

현재 위치에서 함수의 그래디언트값을 계산한 후에 그래디언트의 반대 방향으로 움직이는 방법이다.



그래디언트는 접선의 기울기로 이해해도 됩니다. 접선의 기울기가 양수이면 반대로 w 를 감소시킵니다.



기울기가 양수가 음수거나 간에 빼주면 된다.

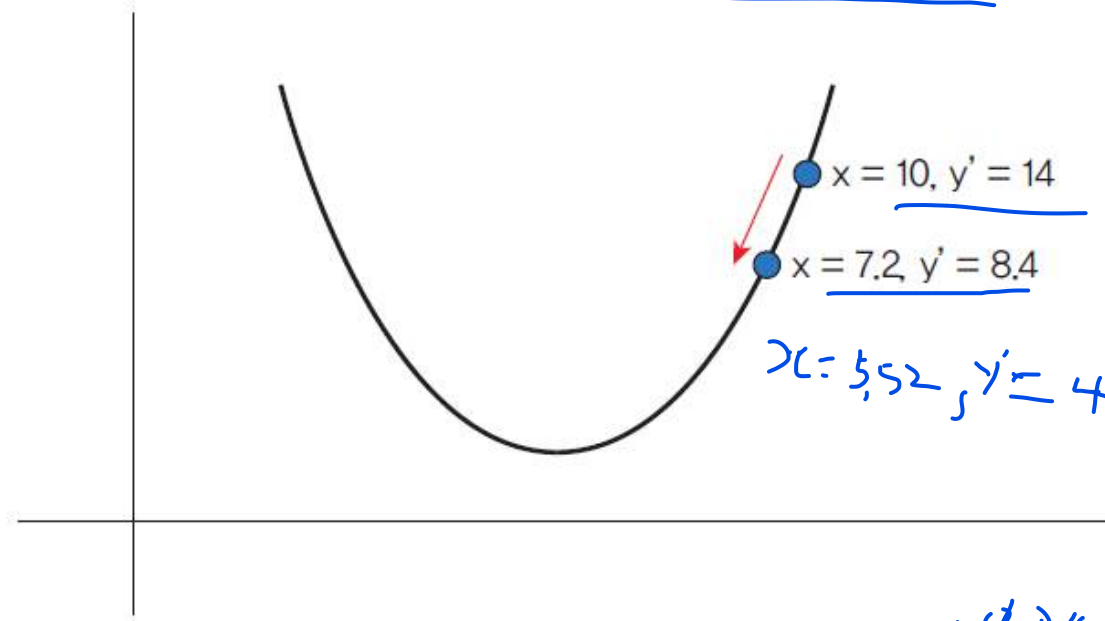
그림 6-11 경사 하강법



Lab: 경사하강법의 실습

- 손실 함수 $y = (x - 3)^2 + 10$
- 그래디언트: $y' = 2x - 6$, 학습률: 0.2
- $x=10, y' = 14$, 새로운 $x = 10 - 0.2 * 14 = 7.2$, 다음 x 는? (lab)

$$y' = 14.4 - 6 = 8.4$$



$$x = 5.52, y' = 4.04$$

$$10 - 0.2 \times 14 = 7.2$$

$$(4.2)^2 = 16.8$$

$$7.2 - 0.2 \times 8.4$$

$$7.2 - 1.68 = 5.52$$

그림 6-12 그래디언트의 계산

$$14.4 - 6$$

경사 하강법 프로그래밍

```
x = 10
learning_rate = 0.2
precision = 0.00001
max_iterations = 100

# 손실함수를 람다식으로 정의한다.
loss_func = lambda x: (x-3)**2 + 10

# 그래디언트를 람다식으로 정의한다. 손실함수의 1차 미분값이다.
gradient = lambda x: 2*x-6

# 그래디언트 강하법
for i in range(max_iterations):
    x = x - learning_rate * gradient(x)
    print("손실함수값(", x, ")=", loss_func(x))

print("최소값 = ", x)
```

실행 결과

```
손실함수값( 7.199999999999999 )= 27.639999999999993  
손실함수값( 5.52 )= 16.350399999999997  
손실함수값( 4.512 )= 12.286143999999998  
손실함수값( 3.9071999999999996 )= 10.82301184  
손실함수값( 3.54432 )= 10.2962842624  
...  
손실함수값( 3.0000000000000004 )= 10.0  
최소값 = 3.0000000000000004
```

Q: 프로그램에서 반복 횟수를 증가시켜서 최소값 $x = 3.0$ 을 찾는 지를 확인하라. 반복횟수 몇 에서 이에 도달하는가?

Q

직접

- 다음과 같은 함수의 최소값을 찾는데 경사 하강법을 적용하라. 앞의 예제 프로그램을 참고하여 $x=20$ 부터 시작하고 학습률은 0.01로 한다. 최소값을 갖는 x 는 얼마인가?

$$f(x) = 0.33 * x^3 + 50 * x^2 - 100 * x - 30$$

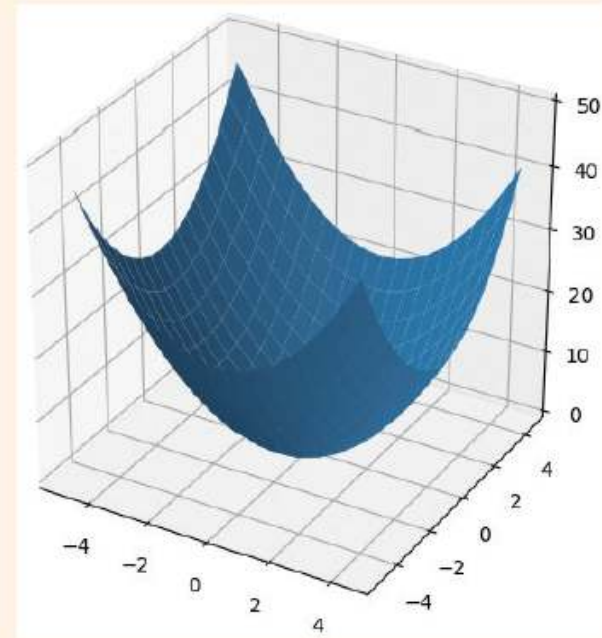
Lab: 2차원 그래디언트 시각화

```
from mpl_toolkits.mplot3d import axes3d
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(-5, 5, 0.5)
y = np.arange(-5, 5, 0.5)
X, Y = np.meshgrid(x, y) # 참고 박스
Z = X**2 + Y**2           # 넘파이 연산

fig = plt.figure(figsize=(6,6))
ax = fig.add_subplot(111, projection='3d')

# 3차원 그래프를 그린다.
ax.plot_surface(X, Y, Z)
plt.show()
```



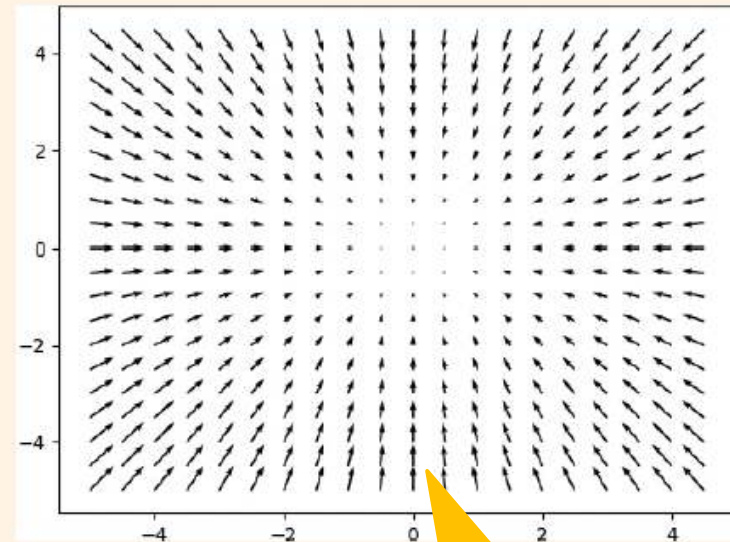
Lab: 2차원 그래디언트 시각화

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(-5,5,0.5)
y = np.arange(-5,5,0.5)
X, Y = np.meshgrid(x,y)
U = -2*X
V = -2*Y

plt.figure()
Q = plt.quiver(X, Y, U, V, units='width')
plt.show()
```

그래디언트의 음수



화살표가 최소값을
가리키고 있음을 알
수 있다.

역전파 학습 알고리즘

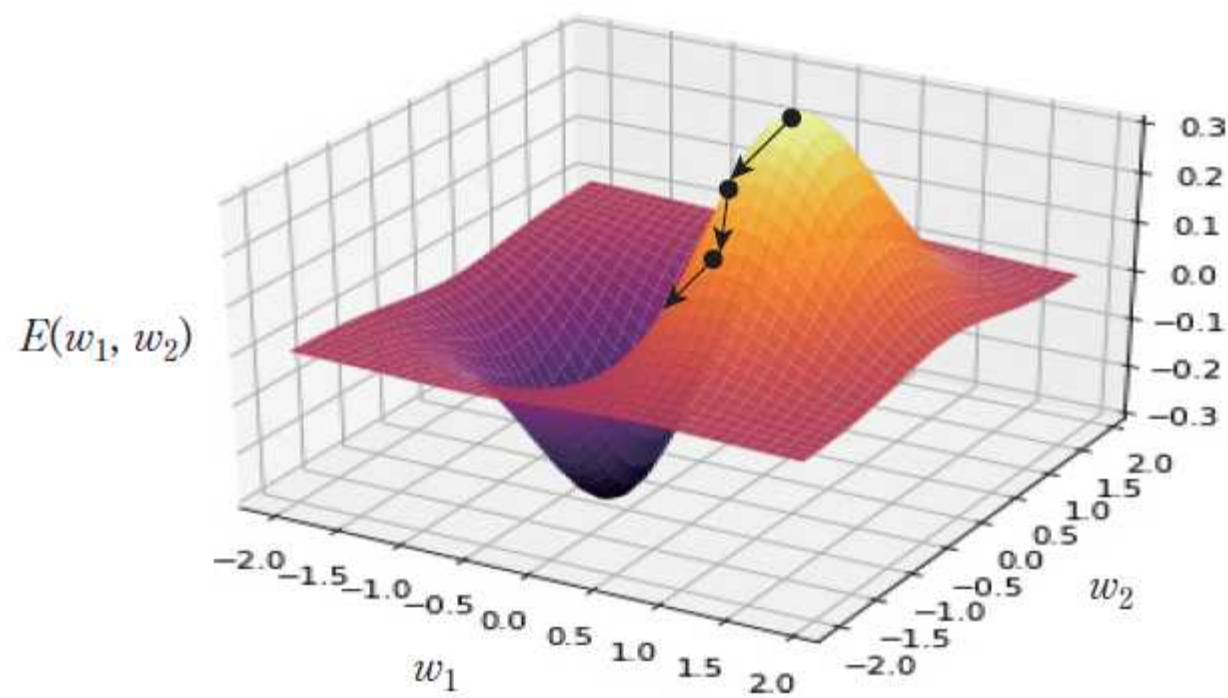
- 역전파 알고리즘은 입력이 주어지면 순방향으로 계산하여 출력을 계산한 후에 실제 출력과 우리가 원하는 출력 간의 오차를 계산한다.
- 이 오차를 역방향으로 전파하면서 오차를 줄이는 방향으로 가중치를 변경한다.

새로운 가중치는 기존 가중치에서 미분해서 그래디언트 곱해서 빼준다..?

$$w(t+1) = w(t) - \eta \frac{\partial E}{\partial w}$$

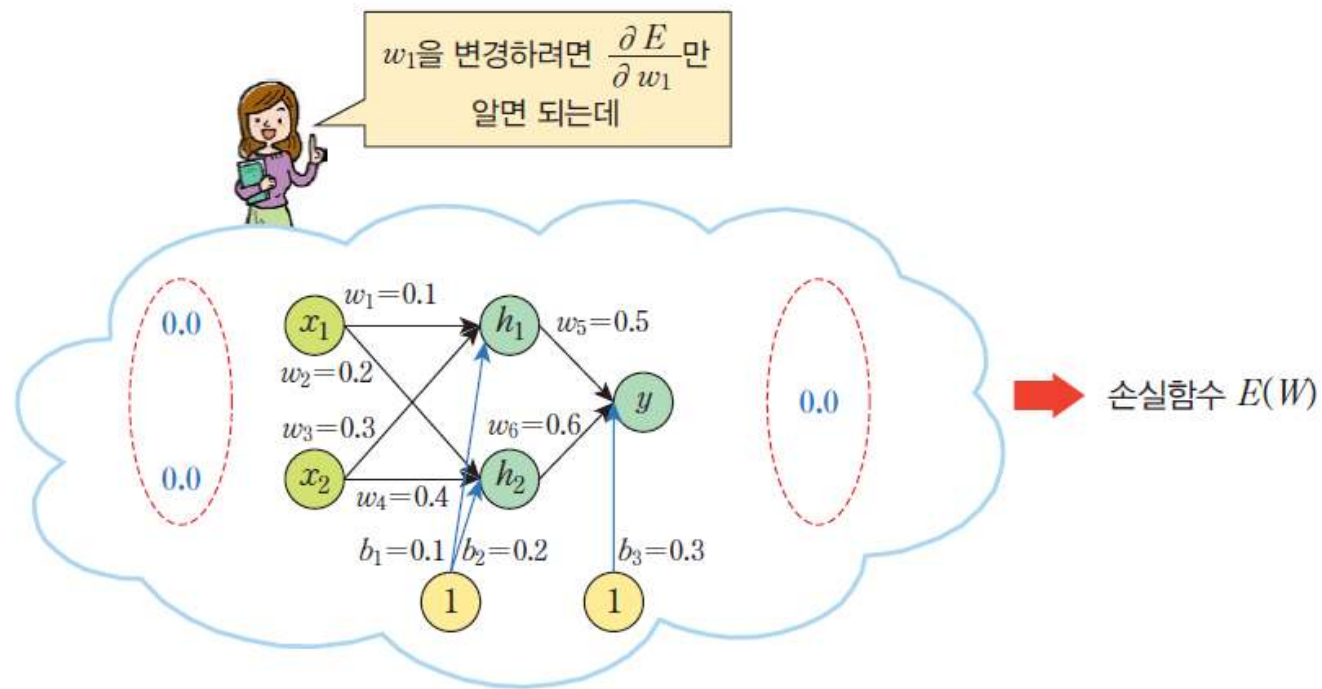
- ① 가중치와 바이어스를 0부터 1 사이의 난수로 초기화한다.
- ② 수렴할 때까지 모든 가중치에 대하여 다음을 반복한다.
- ③ 손실함수 E 의 그래디언트 $\partial E / \partial w$ 을 계산한다.
- ④ $w(t+1) = w(t) - \eta \frac{\partial E}{\partial w}$

역전파 학습 알고리즘



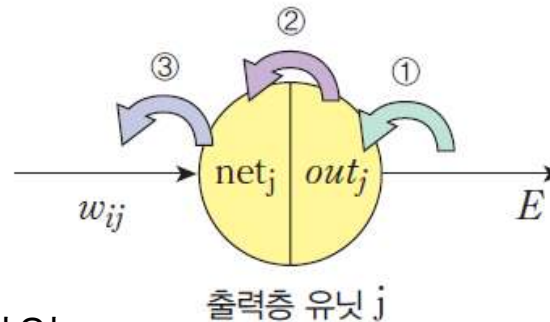
역전파 알고리즘의 유도

- 미분의 체인룰을 이용하여 유도가 가능하다.



출력층 유닛의 경우

$$\frac{\partial E}{\partial w_{ij}} = \underbrace{\frac{\partial E}{\partial out_j}}_{\text{①}} \underbrace{\frac{\partial out_j}{\partial net_j}}_{\text{②}} \underbrace{\frac{\partial net_j}{\partial w_{ij}}}_{\text{③}}$$



다른 노드에서 오는 총 입력의 합이 net j

①, ②가 모여서 델타임

아웃풋이 바뀔 때 타겟에서 실제 출력을 빼면 오차니까 오차의 제곱을 다 더한 것이 에러이다.

① $\frac{\partial E}{\partial out_j} = \frac{\partial}{\partial out_j} \sum \frac{1}{2} (target_k - out_k)^2 = out_j - target_j$ 아웃에 대하여 미분

② $\frac{\partial out_j}{\partial net_j} = \frac{\partial f(net_j)}{\partial net_j} = f'(net_j)$ 활성화 함수를 미분한 거

③ $\frac{\partial net_j}{\partial w_{ij}} = \frac{\partial}{\partial w_{ij}} (\sum_{k=0} w_{kj} out_k) = \frac{\partial}{\partial w_{ij}} w_{ij} out_i = out_i$ i노드에서 연결된 가중치가 바뀔 때 총 입력이 어떻게 바뀌냐

유닛의 출력값 변환에 따른 오차의 변화율이다.

입력합의 변화에 따른 유닛 j의 출력 변화율이다. 활성화 함수의 미분값이다.

가중치의 변화에 따른 net_j의 변화율이라고 할 수 있다.

wij가 바뀔 때 입력의 총합이 어떻게 바뀌느냐

$$\therefore \frac{\partial E}{\partial w_{ij}} = \text{①} \times \text{②} \times \text{③} = (out_j - target_j) \times f'(net_j) \times out_i$$

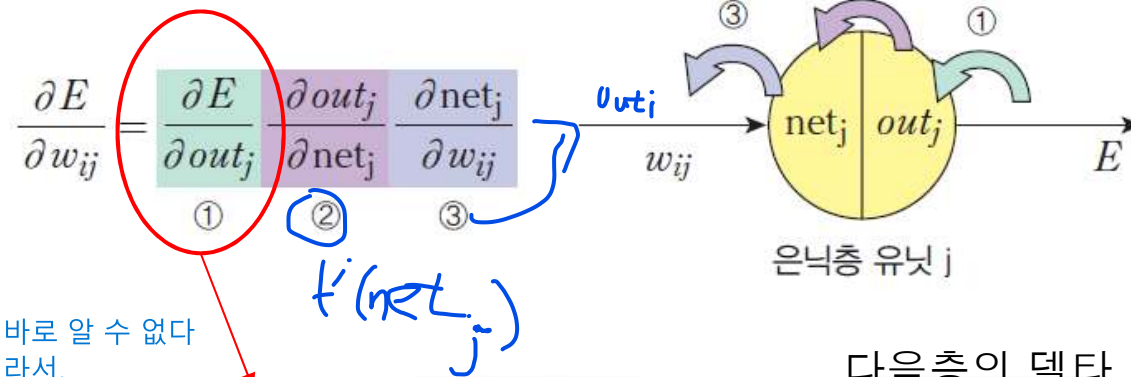
내 앞에 있는 i에서 오는 출력

출력층에 있는 j번째 노드

체인룰을 적용한다.
시그모이드 함수를 가정

은닉층 유닛의 경우

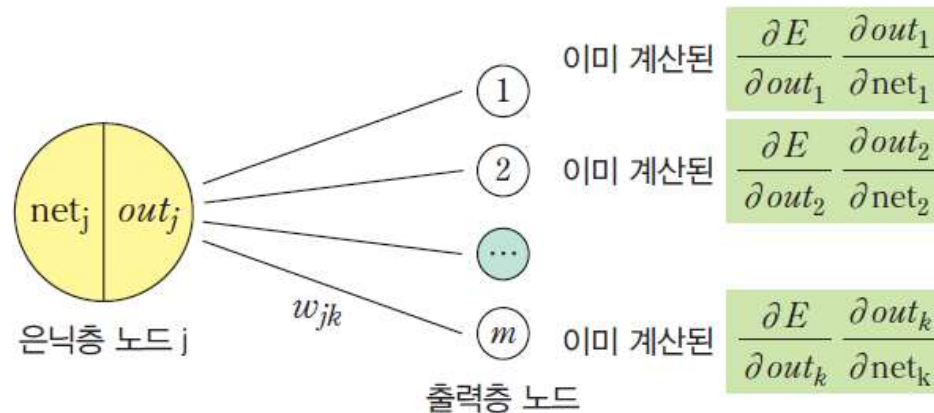
- ①, ②가 모여서 델타임
 ①은 다음층의 델타 * w_{jk}



은닉층에서 에러를 바로 알 수 없다
 최종 출력층이 아니라서.

$$\frac{\partial E}{\partial out_j} = \sum_{k \in L} \left(\frac{\partial E}{\partial out_k} \frac{\partial out_k}{\partial net_k} \frac{\partial net_k}{\partial out_j} \right) = \sum_{k \in L} \left(\frac{\partial E}{\partial out_k} \frac{\partial out_k}{\partial net_k} w_{jk} \right)$$

다음층의 델타



그런데 은닉층 유닛의
 오차 δ 는 어떻게 계산하지?

이것도 체인룰이다!

델타

- $\frac{\partial E}{\partial out_k} \frac{\partial out_k}{\partial net_k}$ 은 여러 문헌에서 델타라는 이름으로 불리는 값이다.
- 가중치가 보는 유닛 k에서의 “오차”라고 생각해도 된다.
- 이 델타가 신경망을 통하여 역전파된다.

역전파 알고리즘 정리

- 결론적으로 그래디언트는 델타에 유닛의 출력값을 곱하면 구할 수 있다.
- 델타는 신경망의 레이어에 따라서 다음과 같이 구분하여서 계산할 수 있다.

$$\frac{\partial E}{\partial w_{ij}} = \delta_j \text{out}_i \quad \text{여기서} \quad \delta_j = \begin{cases} (out_j - t_j)f'(net_j) & j \text{가 출력층 유닛이면} \\ \left(\sum_k w_{jk} \delta_k\right)f'(net_j) & j \text{가 은닉층 유닛이면} \end{cases}$$

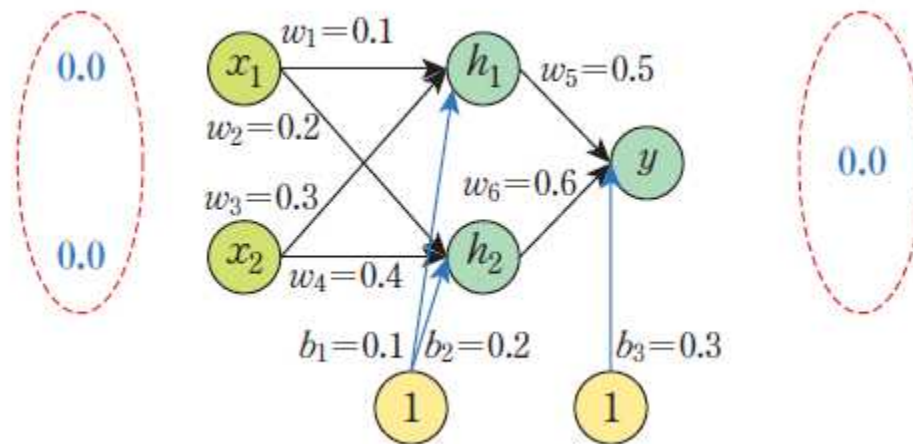
나의 그래디언트 값을 알려면
k가 어디있죠? 나 다음이죠?

거꾸로 뒤에 있는 거를 가져와야지 내꺼를 계산할 수 있어서 이름이 역전파이다.

3 =

아무 노드의 출력

역전파 알고리즘을 손으로 계산해보자.



- 순방향 패스

$$\begin{aligned} \text{net}_y &= w_5 * \text{out}_{h1} + w_6 * \text{out}_{h2} + b_3 \\ &= 0.5 * 0.524979 + 0.6 * 0.549834 + 0.3 = \underline{0.89239} \end{aligned}$$

$$\text{out}_y = \frac{1}{1 + e^{-\text{net}_y}} = \frac{1}{1 + e^{-0.89239}} = 0.709383$$

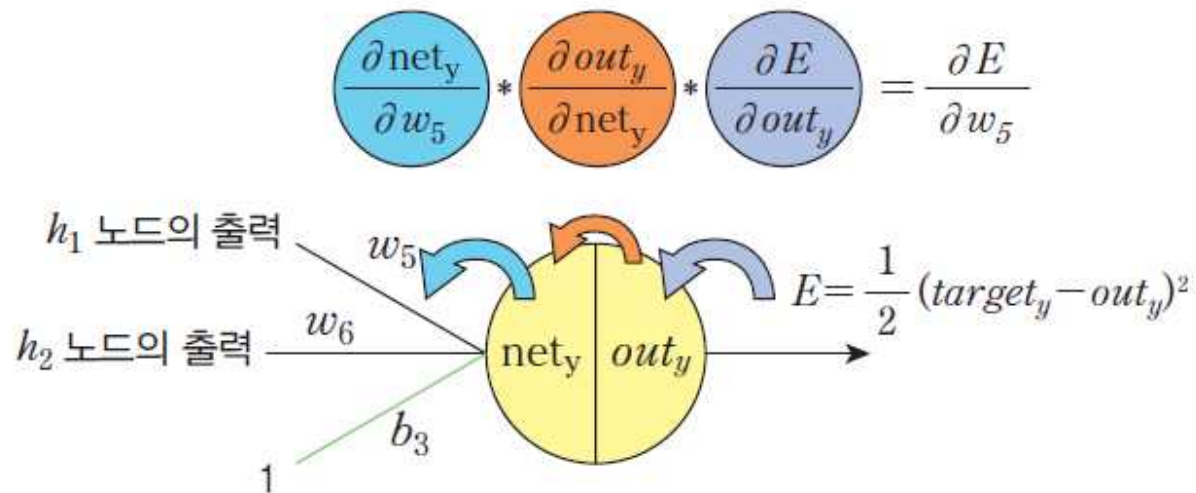
앞절 참조!

역전파 알고리즘을 손으로 계산해보자.

- 총오차 계산

$$E = \frac{1}{2}(target_y - out_y)^2 = \frac{1}{2}(0.00 - 0.709383)^2 = 0.251612$$

- $\frac{\partial E}{\partial w_5}$ 만 계산해보자.



경사하강법 적용

- ① $\frac{\partial E}{\partial out_y} = 2 * \frac{1}{2} (target_y - out_y)^{2-1} * (-1) = (out_y - target_y)$
 $= (0.709383 - 0.00) = 0.709383$
- ② $\frac{\partial out_y}{\partial net_y} = f'(out_y) = out_y * (1 - out_y) = 0.709383 * (1 - 0.709383) = \underline{0.206158}$
- ③ $net_y = w_5 * out_{h1} + w_6 * out_{h2} + b_3 * 1$
 $\frac{\partial net_y}{\partial w_5} = 1 * out_{h1} + 0 + 0 = 0.524979$



$$\frac{\partial E}{\partial w_5} = \frac{\partial E}{\partial out_y} \frac{\partial out_y}{\partial net_y} \frac{\partial net_y}{\partial w_5}$$

$$= 0.709383 * 0.206158 * 0.524979 = \underline{0.076775}$$

$$w_5(t+1) = w_5(t) + \eta * \frac{\partial E}{\partial w_5} = 0.5 - 0.2 * 0.076775 = 0.484645$$

새로운 가중치는 기존 가중치 - 기존계산한 거 곱하기 0.2

은닉층->출력층의 가중치와 바이어스

$$w_5(t+1) = w_5(t) + \eta * \frac{\partial E}{\partial w_5} = 0.5 - 0.2 * 0.076775 = 0.484645$$

$$w_6(t+1) = 0.583918$$

$$b_3(t+1) = 0.270750$$

가중치도 낮아지게 된다. . 현재 우리가 원하는 출력값은 0이기 때문이다.

바이어스는 기존 값 보다 낮아지게 된다. 따라서 다음 번에는 유닛의 출력을 더 낮게 만들 것이다. 현재 우리가 원하는 출력값은 0이기 때문이다.

입력층->은닉층의 가중치와 바이어스

$$w_1(t+1) = w_1(t) + \eta * \frac{\partial E}{\partial w_1} = 0.10 - 0.2 * \underbrace{0.0}_{\text{입력}} = 0.10$$

$$w_2(t+1) = 0.2, \quad w_3(t+1) = 0.3, \quad w_4(t+1) = 0.4$$

입력값이 0이어서 가중치는 변경되지 않았다(이점은 퍼셉트론과 유사하다. 입력이 0이면 가중치를 아무리 바꿔도 무슨 소용인가?).

$$\underline{b_1(t+1) = 0.096352}, \quad \underline{b_2(t+1) = 0.195656}$$

바이어스만 학습이 된다.

이런 경우에는 바이어스가 큰 역할을 한다(이래서 바이어스는 반드시 있어야 한다) **바이어스는 기존 값 보다 낮아지게 된다.**
따라서 다음 번에는 유닛의 출력을 더 낮게 만들 것이다. 현재 우리가 원하는 출력값은 0이기 때문이다.

손실함수 평가

$$E = \frac{1}{2}(target - out_y)^2 = \frac{1}{2}(0.00 - 0.709383)^2 = 0.251612$$



경사하강법 1번 적용

$$E = \frac{1}{2}(target - out_y)^2 = \frac{1}{2}(0.00 - 0.699553)^2 = 0.244687$$



경사하강법 10000번 적용

$$E = \frac{1}{2}(target - out_y)^2 = \frac{1}{2}(0.00 - 0.005770)^2 = 0.000016$$

오차가 크게 줄었다.

넘파이를 이용하여 MLP 구현

- 넘파이의 기능을 이용하면 모든 것을 행렬과 벡터로 표시할 수 있다.
- 행렬을 이용하면 동시에 여러 개의 예제를 동시에 학습시킬 수 있다.
- 역전파할 때는 가중치 행렬을 전치시켜서 사용한다.
- 바이어스는 입력을 1.0으로 고정하고, 이 입력에 붙은 가중치로 생각한다.

넘파이를 이용한 MLP 구현

```
import numpy as np
```

```
# 시그모이드 함수
```

```
def actf(x):
```

```
    return 1/(1+np.exp(-x))
```

$$= \frac{1}{1 + e^{-x}}$$

```
# 시그모이드 함수의 미분치
```

```
def actf_deriv(x):
```

```
    return x*(1-x)
```

```
# 입력유닛의 개수, 은닉유닛의 개수, 출력유닛의 개수
```

```
inputs, hiddens, outputs = 2, 2, 1
```

```
learning_rate=0.2
```

```
# 훈련 샘플과 정답
```

```
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
```

```
T = np.array([[0], [1], [1], [0]])
```

X OR

순방향 전파 구현

```
W1 = np.array([[0.10,0.20], [0.30,0.40]])
W2 = np.array([[0.50],[0.60]])
B1 = np.array([0.1, 0.2])
B2 = np.array([0.3])

# 순방향 전파 계산
def predict(x):
    layer0 = x                                # 입력을 layer0에 대입한다.
    Z1 = np.dot(layer0, W1)+B1                # 행렬의 곱을 계산한다.
    layer1 = actf(Z1)                         # 활성화 함수를 적용한다.
    Z2 = np.dot(layer1, W2)+B2                # 행렬의 곱을 계산한다.
    layer2 = actf(Z2)                         # 활성화 함수를 적용한다.
    return layer0, layer1, layer2
```

오차 역전파 구현

```
# 역방향 전파 계산
def fit():
    global W1, W2, B1, B2
    for i in range(90000):
        for x, y in zip(X, T):
            x = np.reshape(x, (1, -1))
            y = np.reshape(y, (1, -1))

            layer0, layer1, layer2 = predict(x)
            layer2_error = layer2 - y
            layer2_delta = layer2_error * actf_deriv(layer2)
            layer1_error = np.dot(layer2_delta, W2.T)
            layer1_delta = layer1_error * actf_deriv(layer1)

            W2 += -learning_rate * np.dot(layer1.T, layer2_delta)
            W1 += -learning_rate * np.dot(layer0.T, layer1_delta)
            B2 += -learning_rate * np.sum(layer2_delta, axis=0)
            B1 += -learning_rate * np.sum(layer1_delta, axis=0)

# 순방향 계산
# 오차 계산
# 출력층의 델타 계산
# 은닉층의 오차 계산
# 은닉층의 델타 계산
# ④
#
# ⑤
```

오차 역전파 구현

```
def test():  
    for x, y in zip(X, T):  
        x = np.reshape(x, (1, -1))          # 하나의 샘플을 꺼내서 2차원 행렬로 만든다.  
        layer0, layer1, layer2 = predict(x) # 출력층의 값을 출력해본다.  
        print(x, y, layer2)  
  
fit()  
test()
```

순방향 결과 만드는 함수

```
[[0 0]] [0] [[0.00835708]]  
[[0 1]] [1] [[0.99196032]]  
[[1 0]] [1] [[0.98974873]]  
[[1 1]] [0] [[0.00836107]]
```

xor을 역전파가 해결했다.

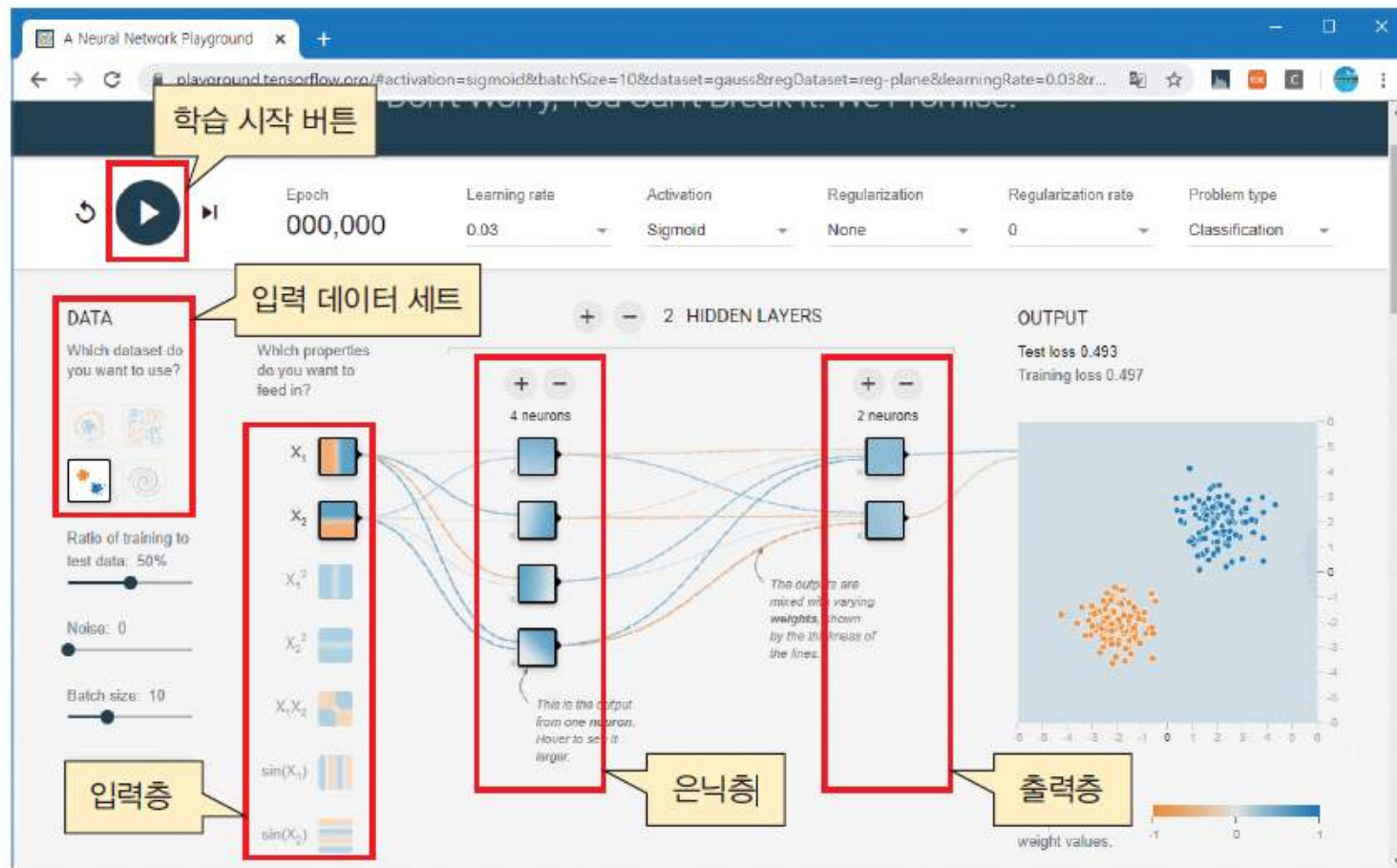
Q

- 앞의 넘파이를 이용하여 **MLP**를 구현한 코드를 사용하여서 **AND** 연산을 학습시켜보라.

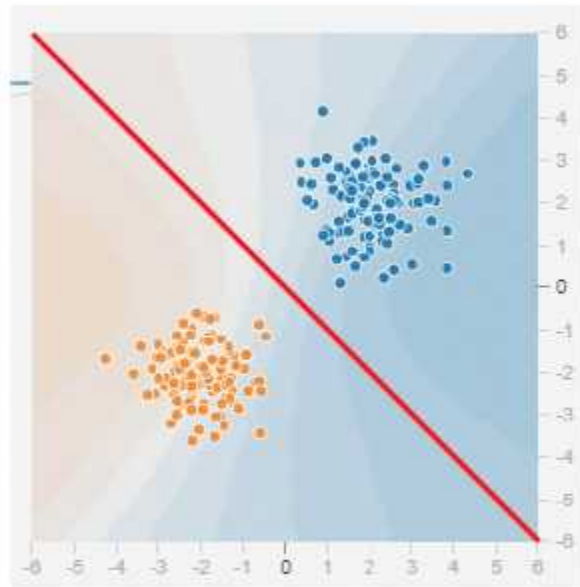
구글의 플레이그라운드

- 사이트(<https://playground.tensorflow.org>)
- 텐서 플로우 플레이그라운드는 자바 스크립트로 작성된 웹 애플리케이션으로 웹 브라우저에서 실행
- 이 사이트에서는 사용자가 딥러닝 모델을 구성하고 여러 가지 매개변수를 조정하면서 실험할 수 있는 기능을 제공한다.

구글의 플레이그라운드

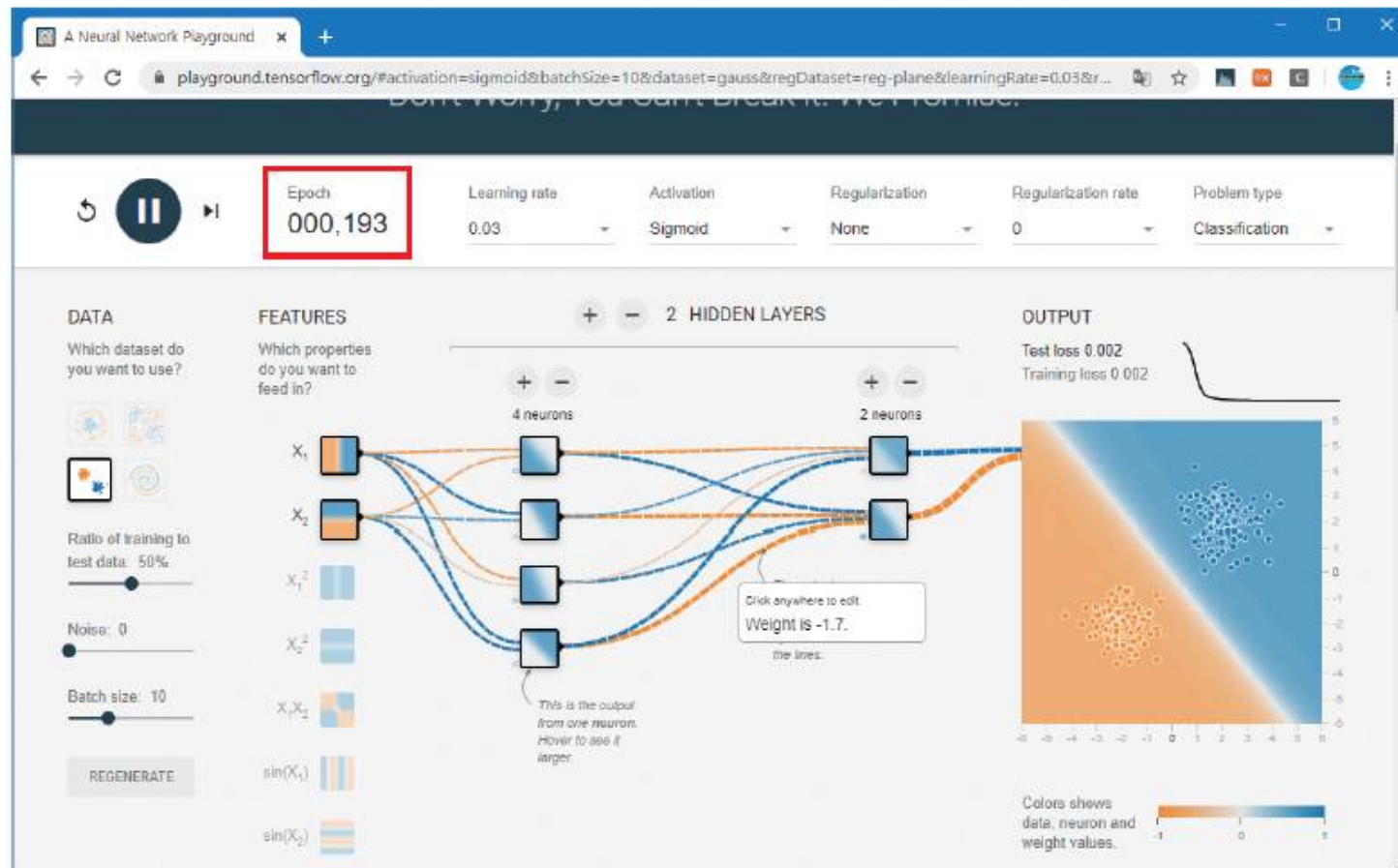


선형 분리 가능한 입력 데이터

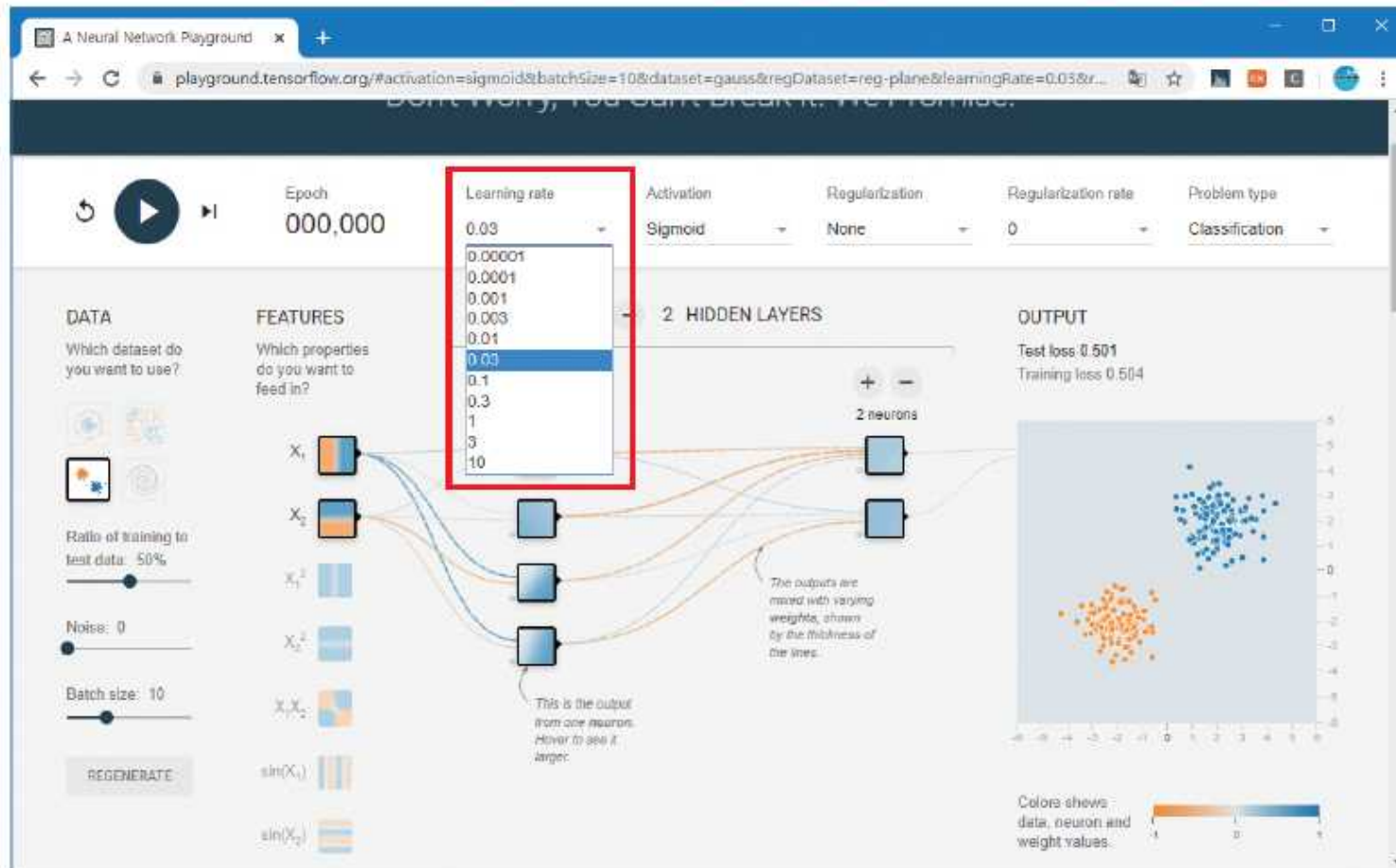


$$w_1x_1 + w_2x_2 + b = 0$$

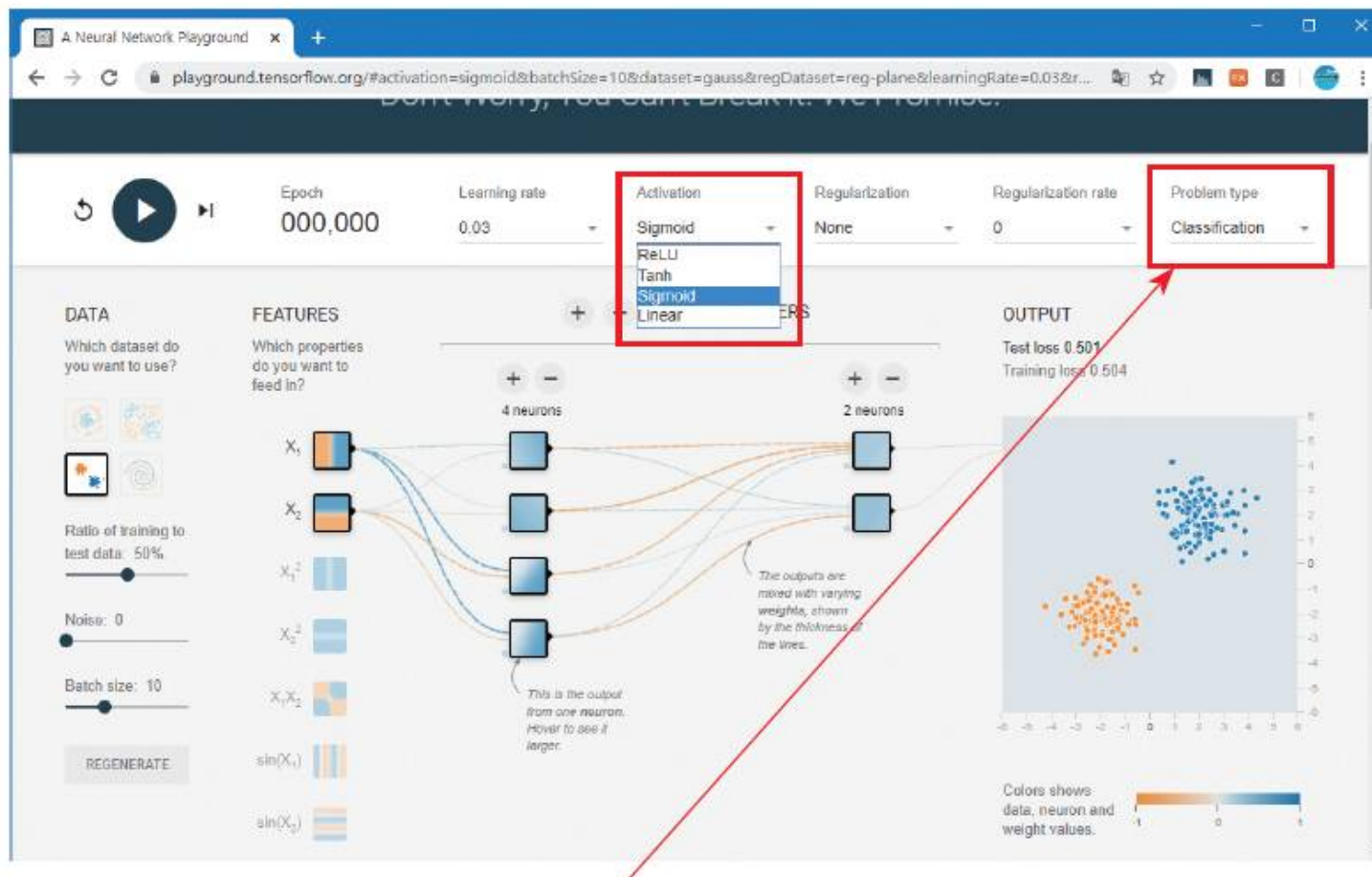
에포크



학습률

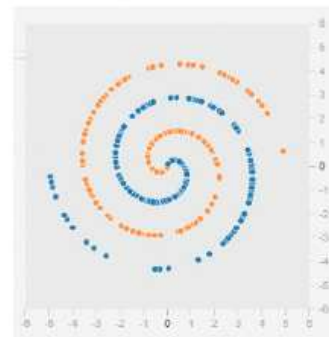
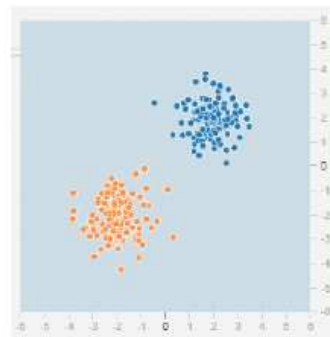
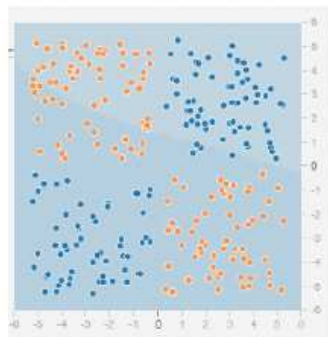
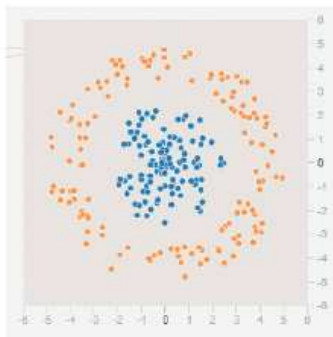


활성화 함수 선택

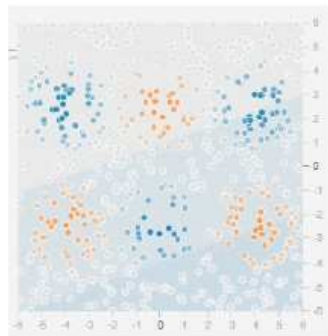
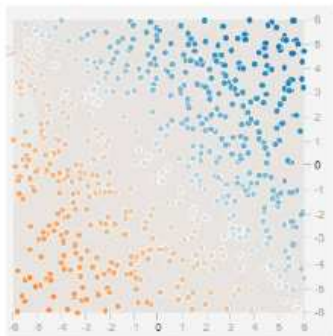


문제 유형

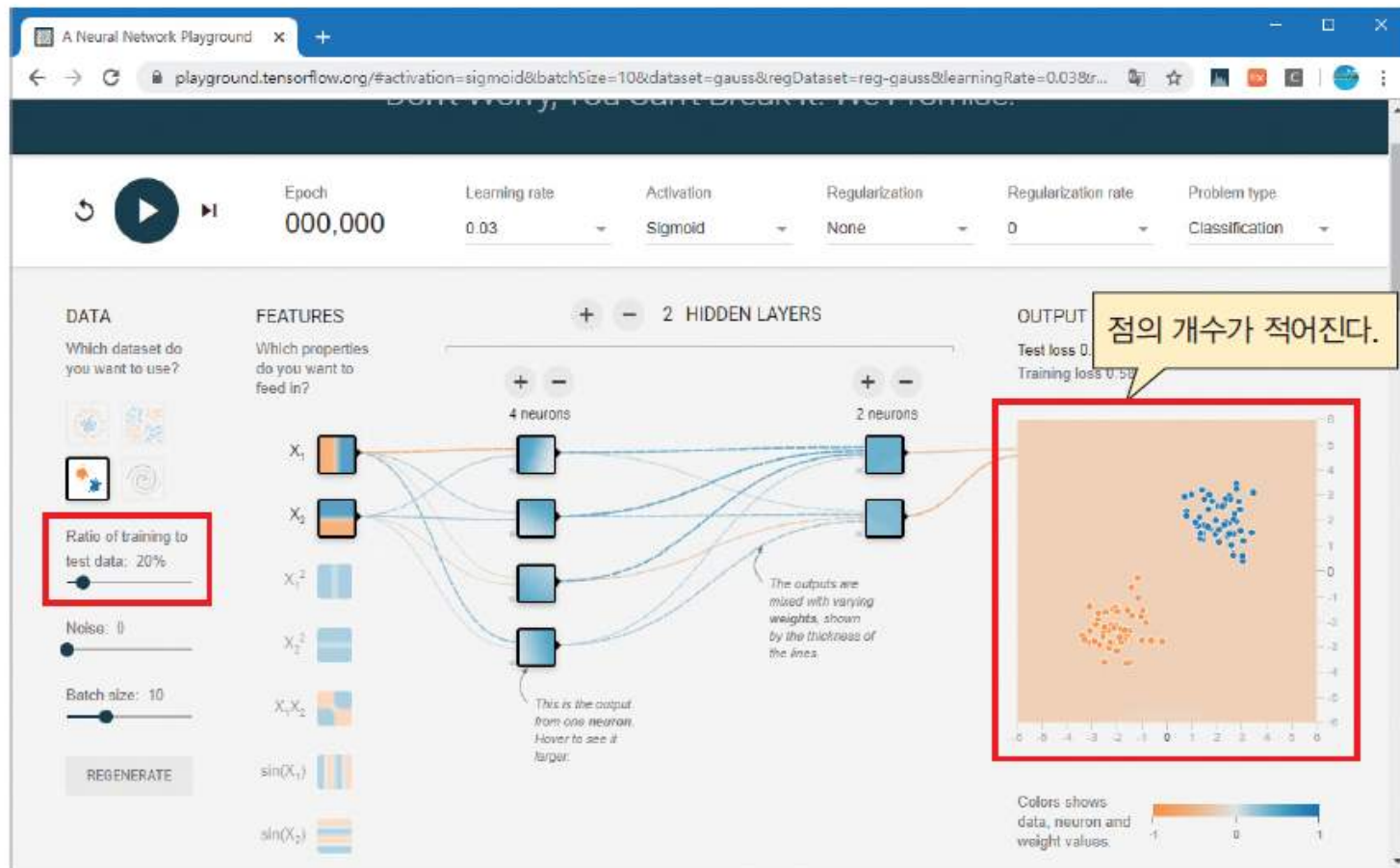
- 분류 문제



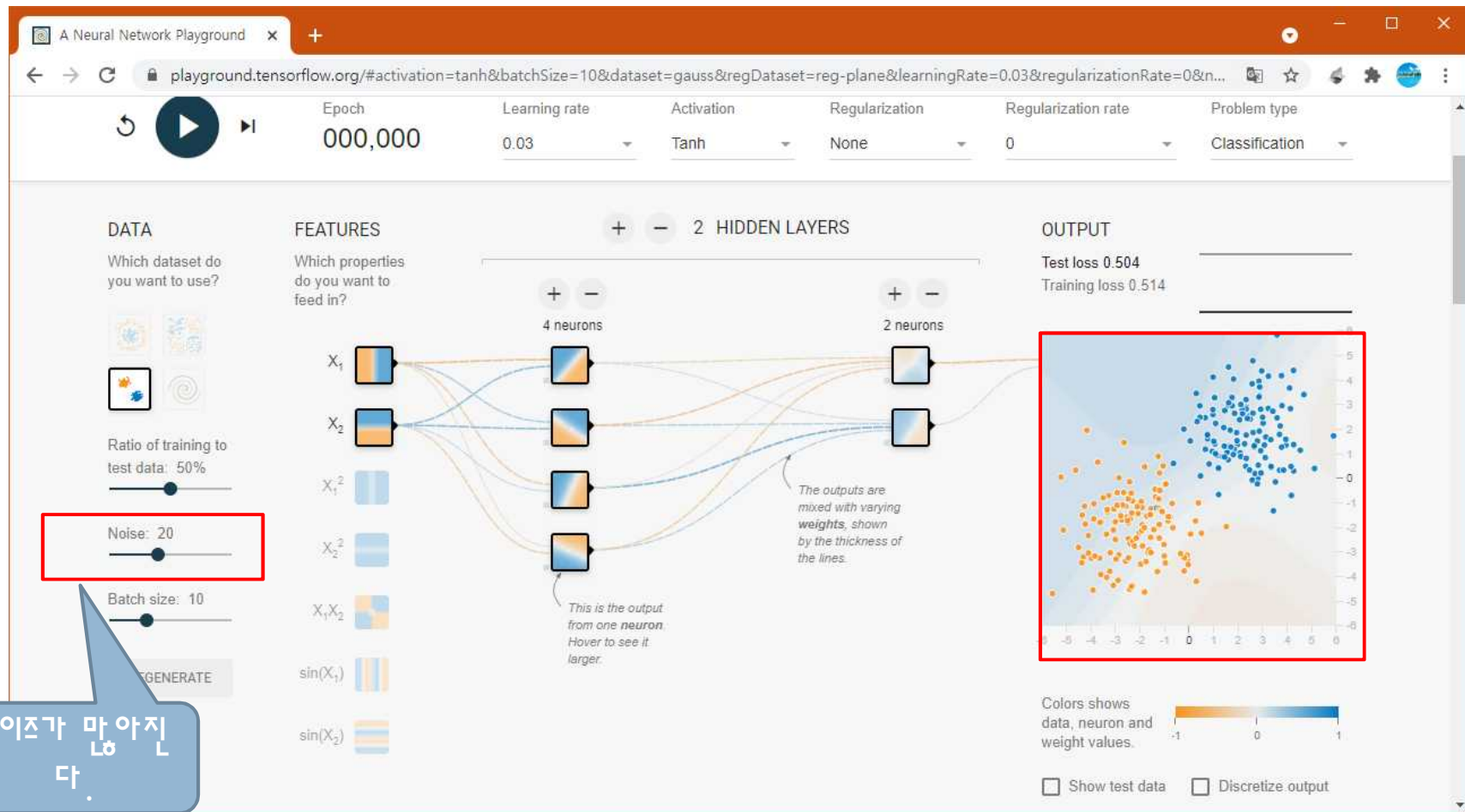
- 회귀 문제



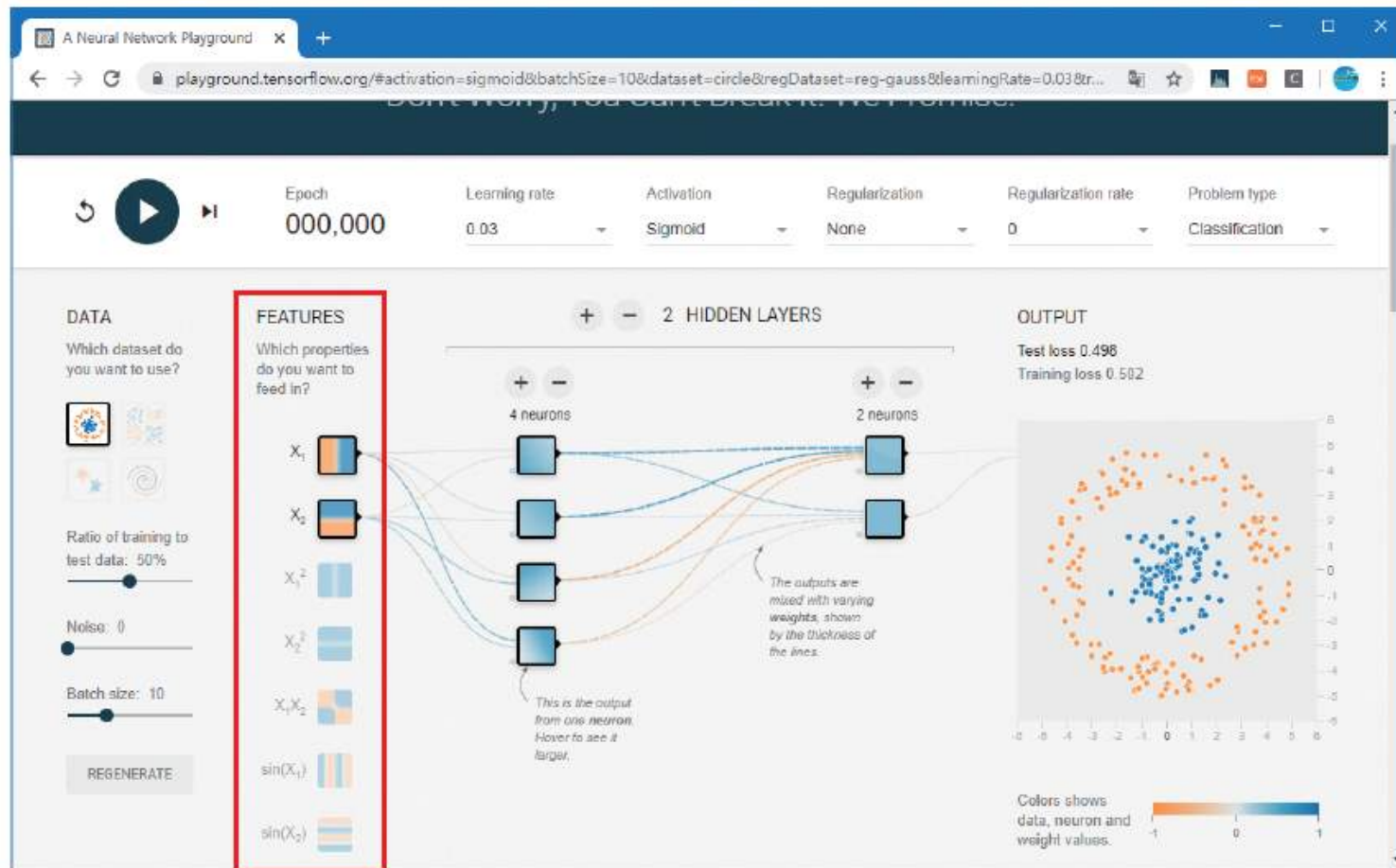
학습 데이터와 테스트 데이터의 비율



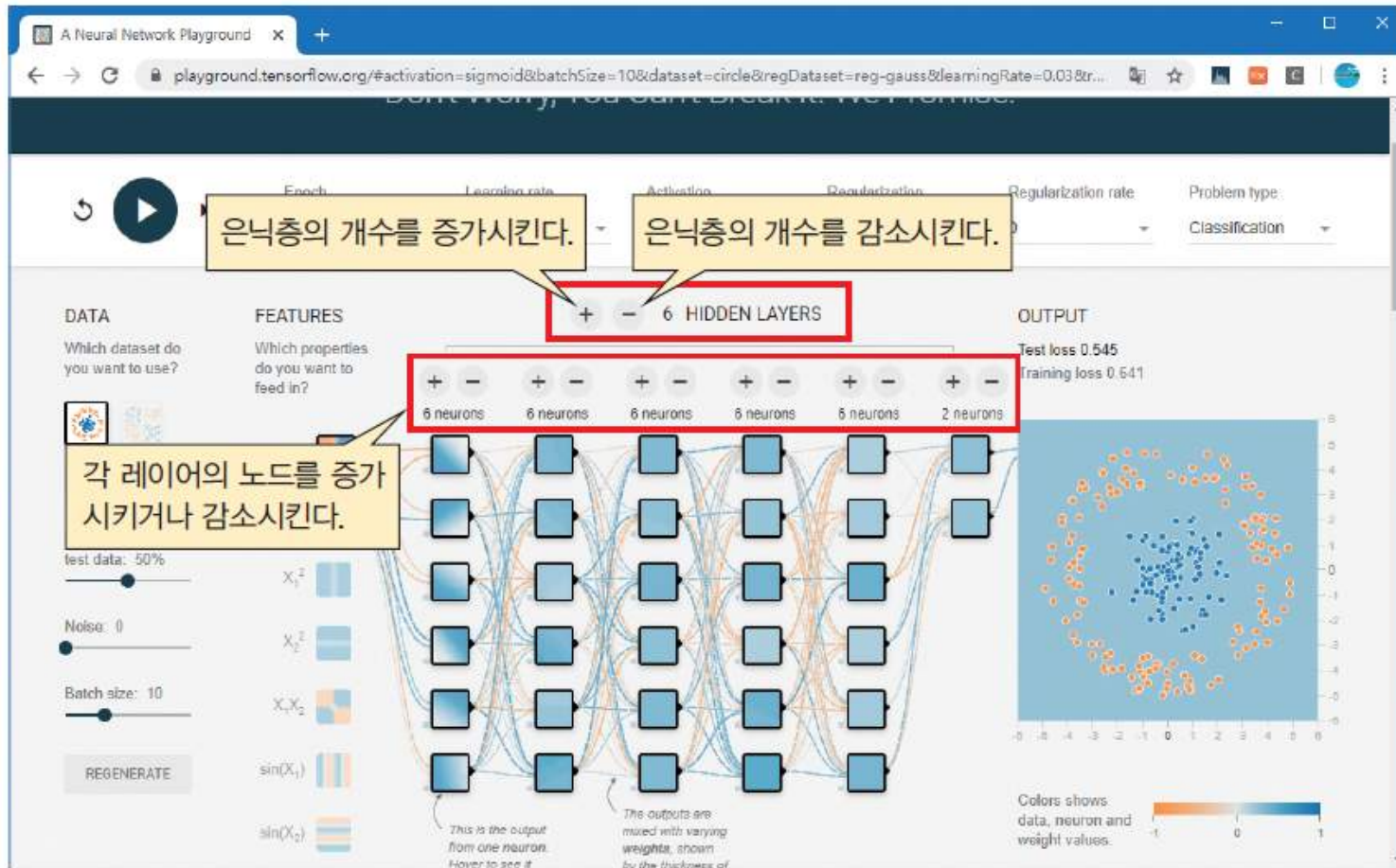
노이즈 비율



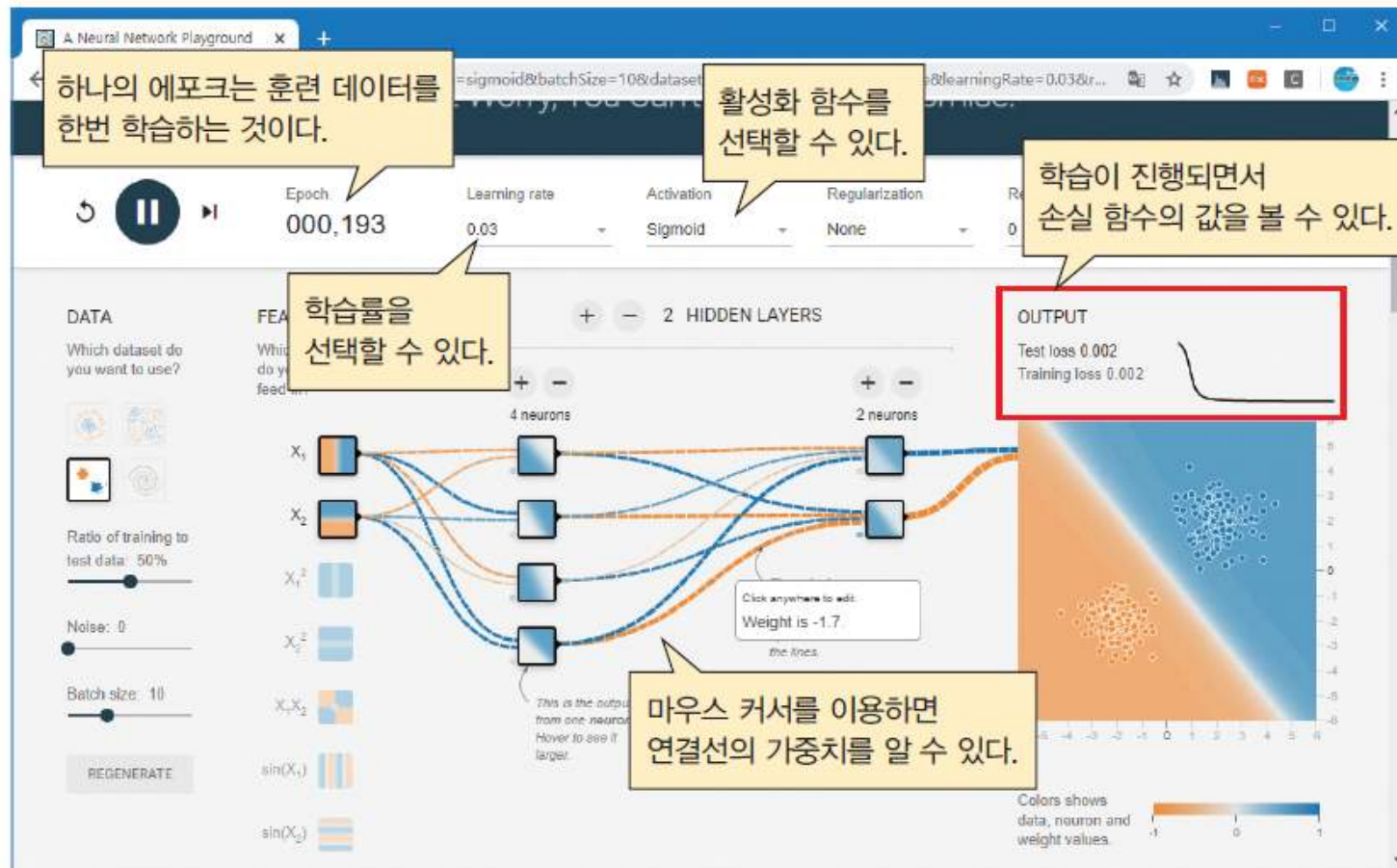
입력 특징 선택



은닉층 추가하기



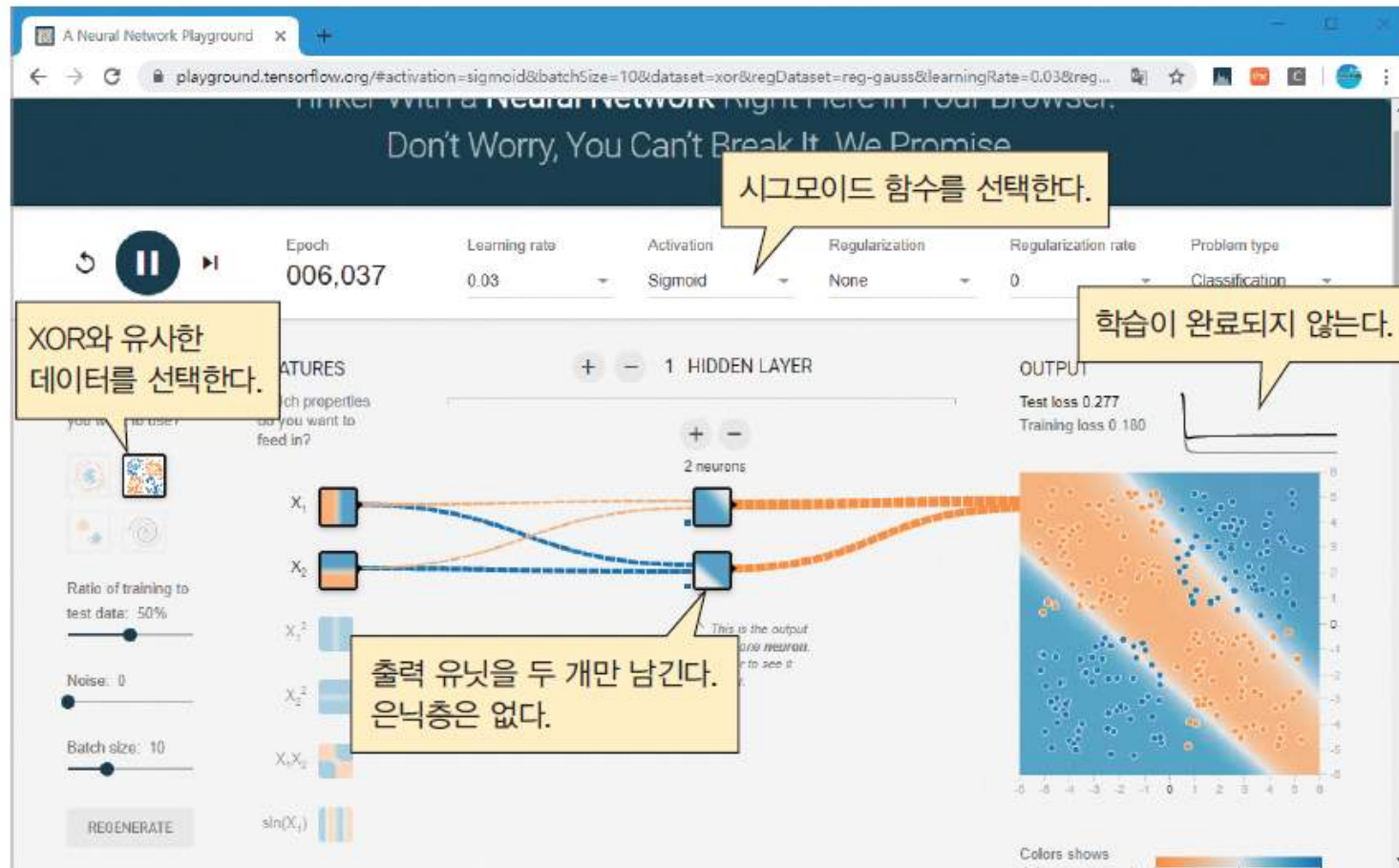
학습 시작



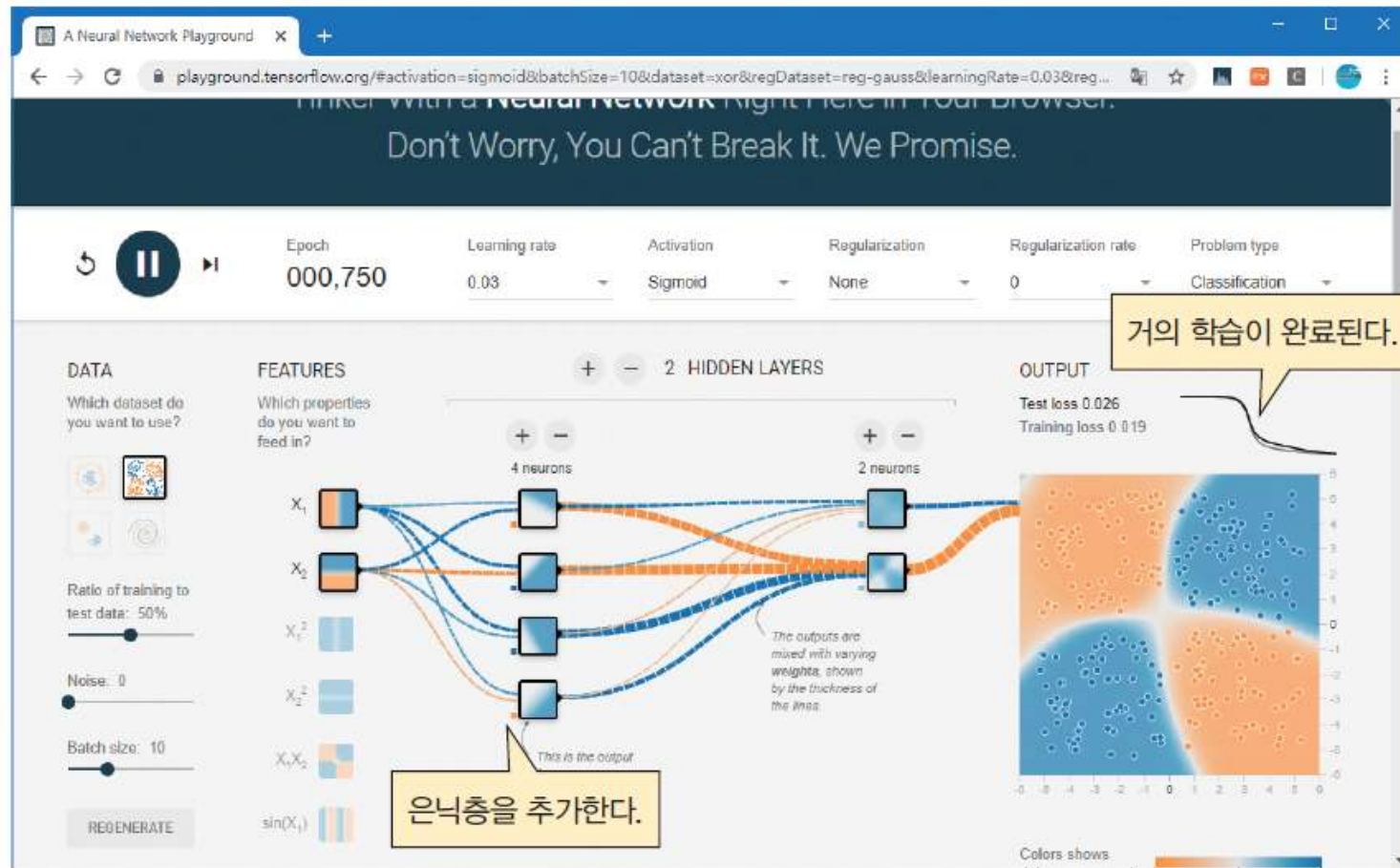
학습 완료



은닉층 없이 분류 실습



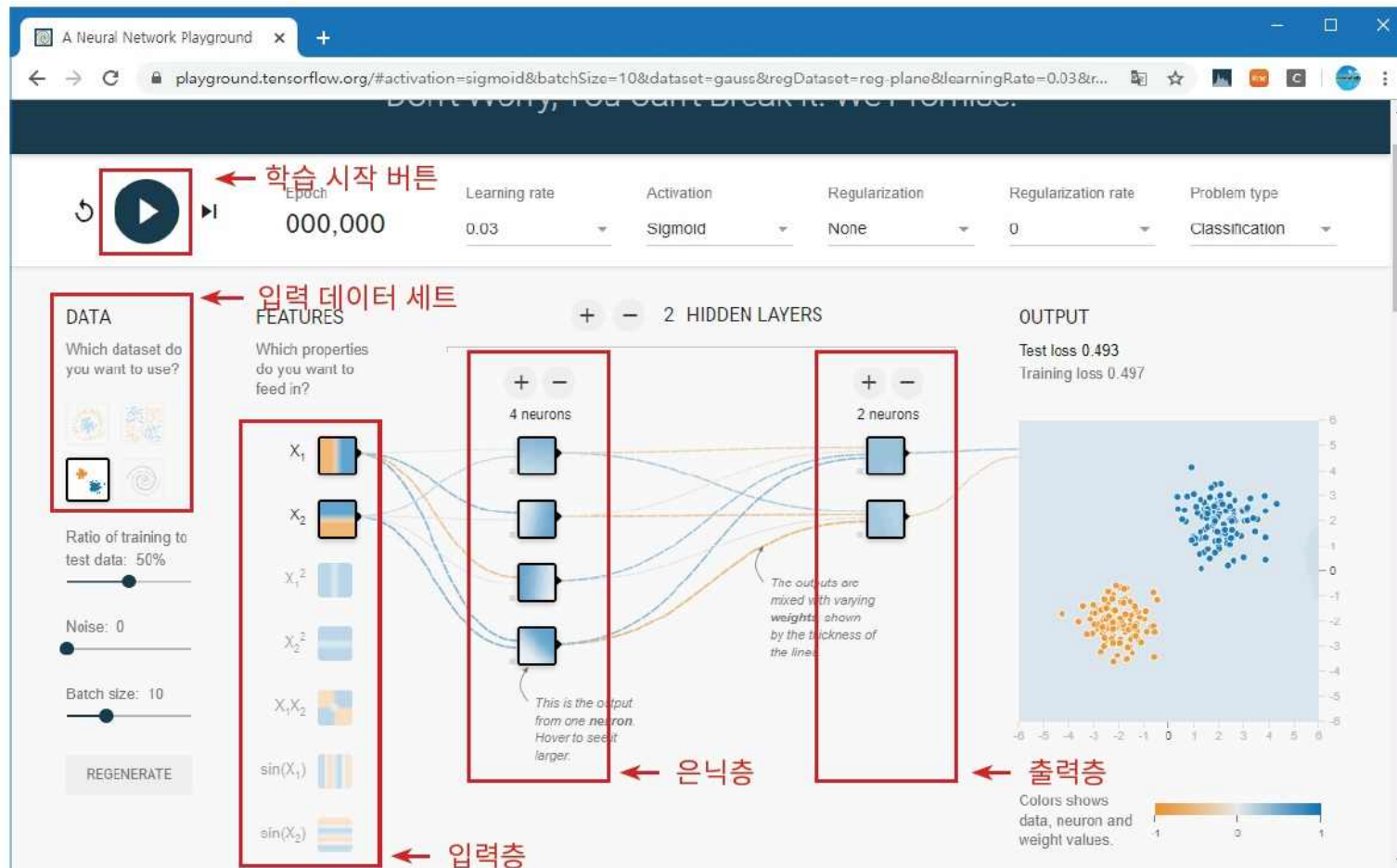
은닉층을 추가한 실습



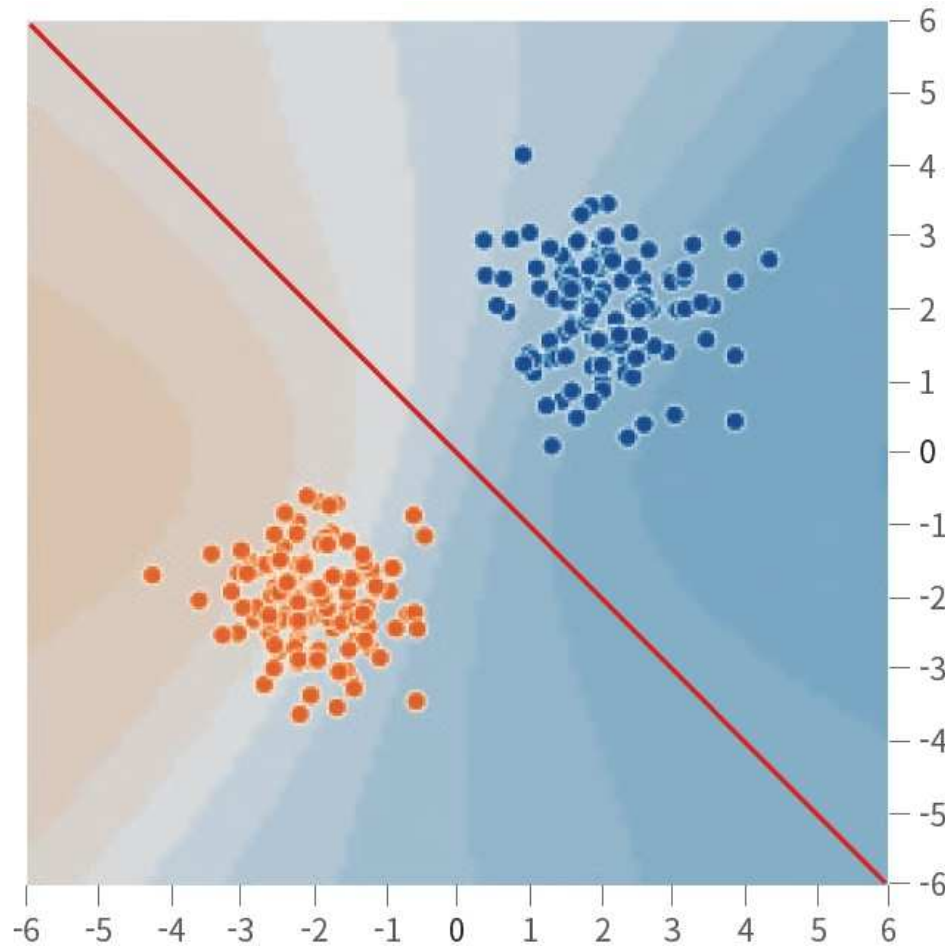
07 구글의 플레이그라운드

- 사이트(<https://playground.tensorflow.org>)
- 텐서 플로우 플레이그라운드는 자바 스크립트로 작성된 웹 애플리케이션으로 웹 브라우저에서 실행
- 이 사이트에서는 사용자가 딥러닝 모델을 구성하고 여러 가지 매개변수를 조정하면서 실험할 수 있는 기능을 제공

구글의 플레이그라운드

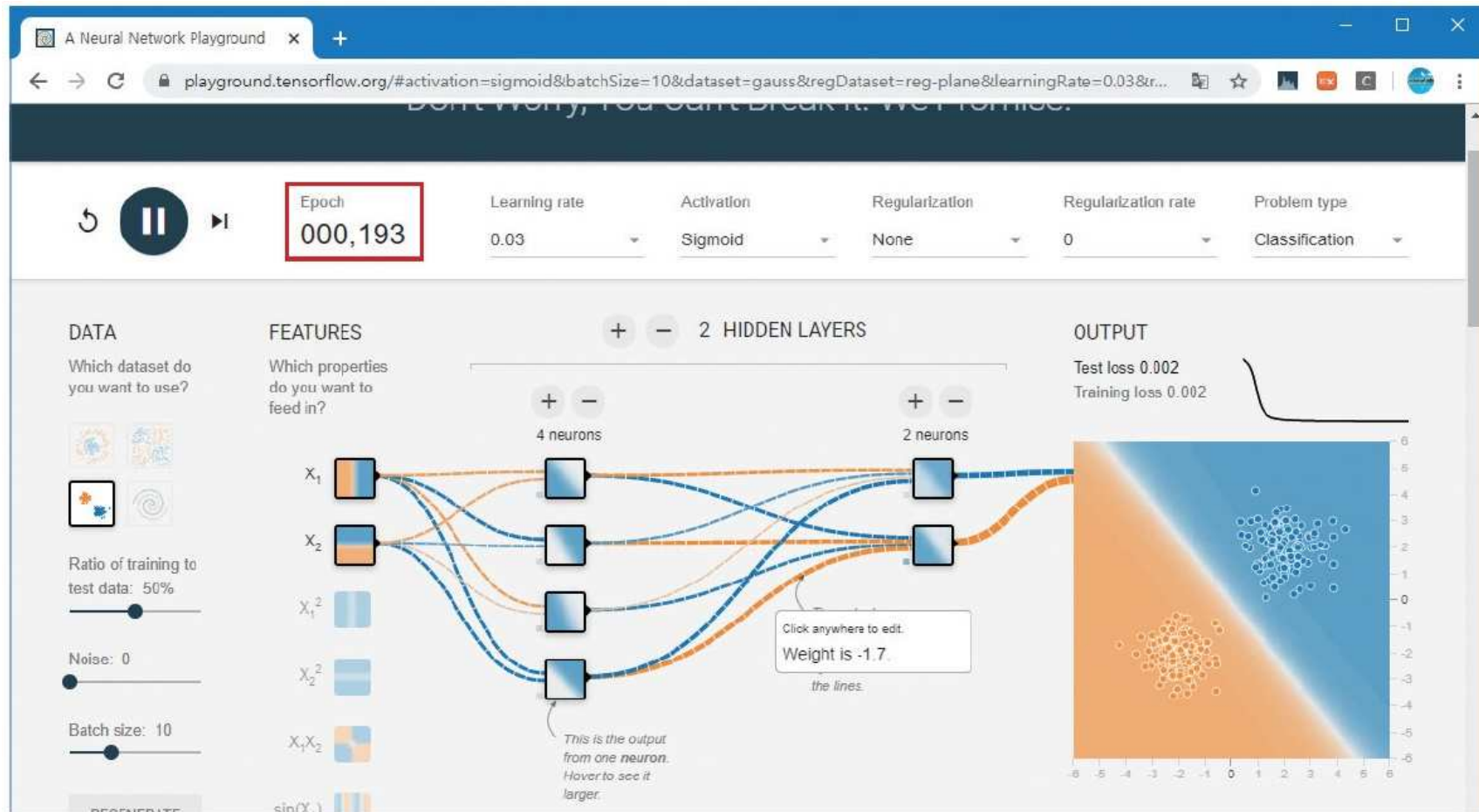


선형 분리 가능한 입력 데이터

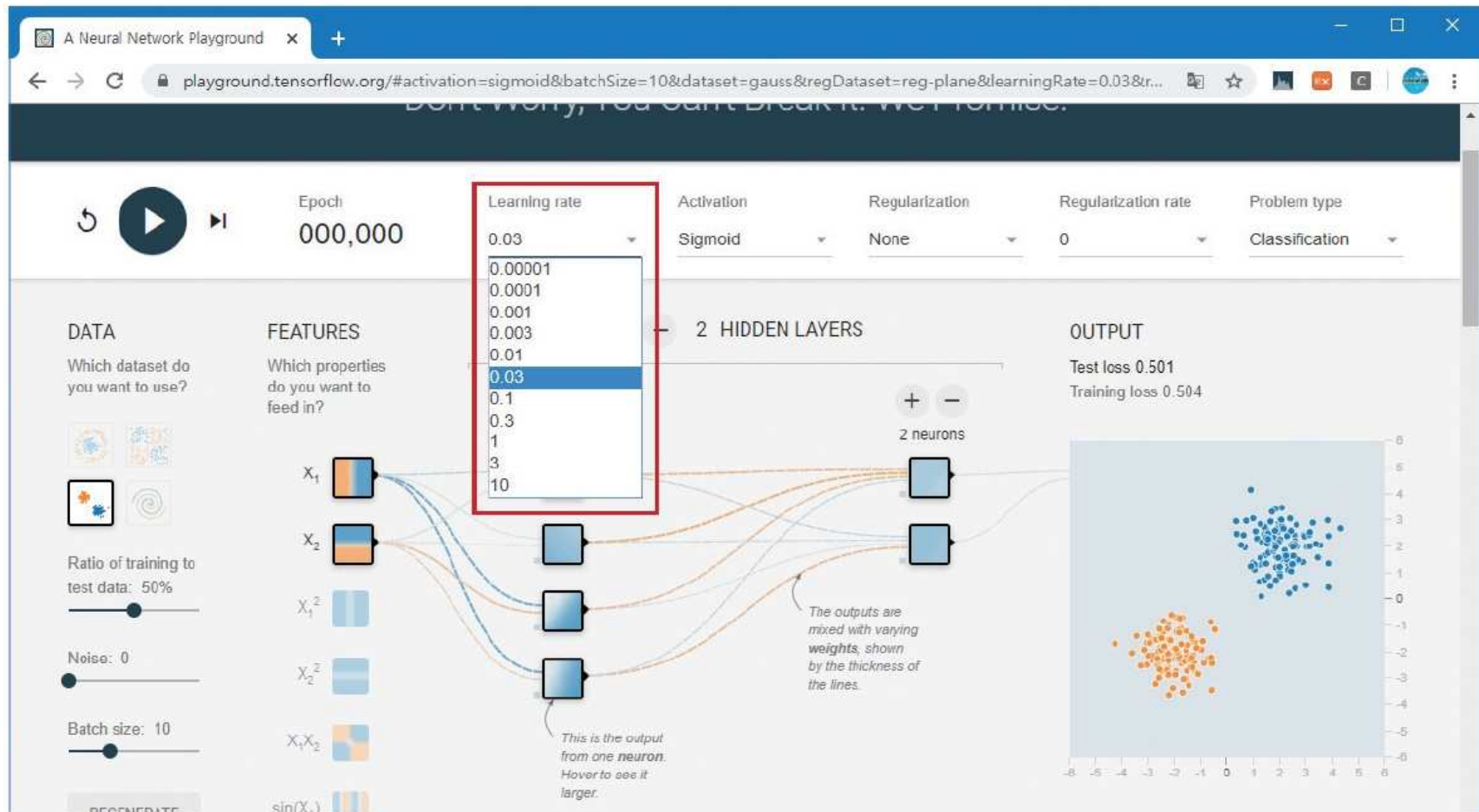


$$w_1x_1 + w_2x_2 + b = 0$$

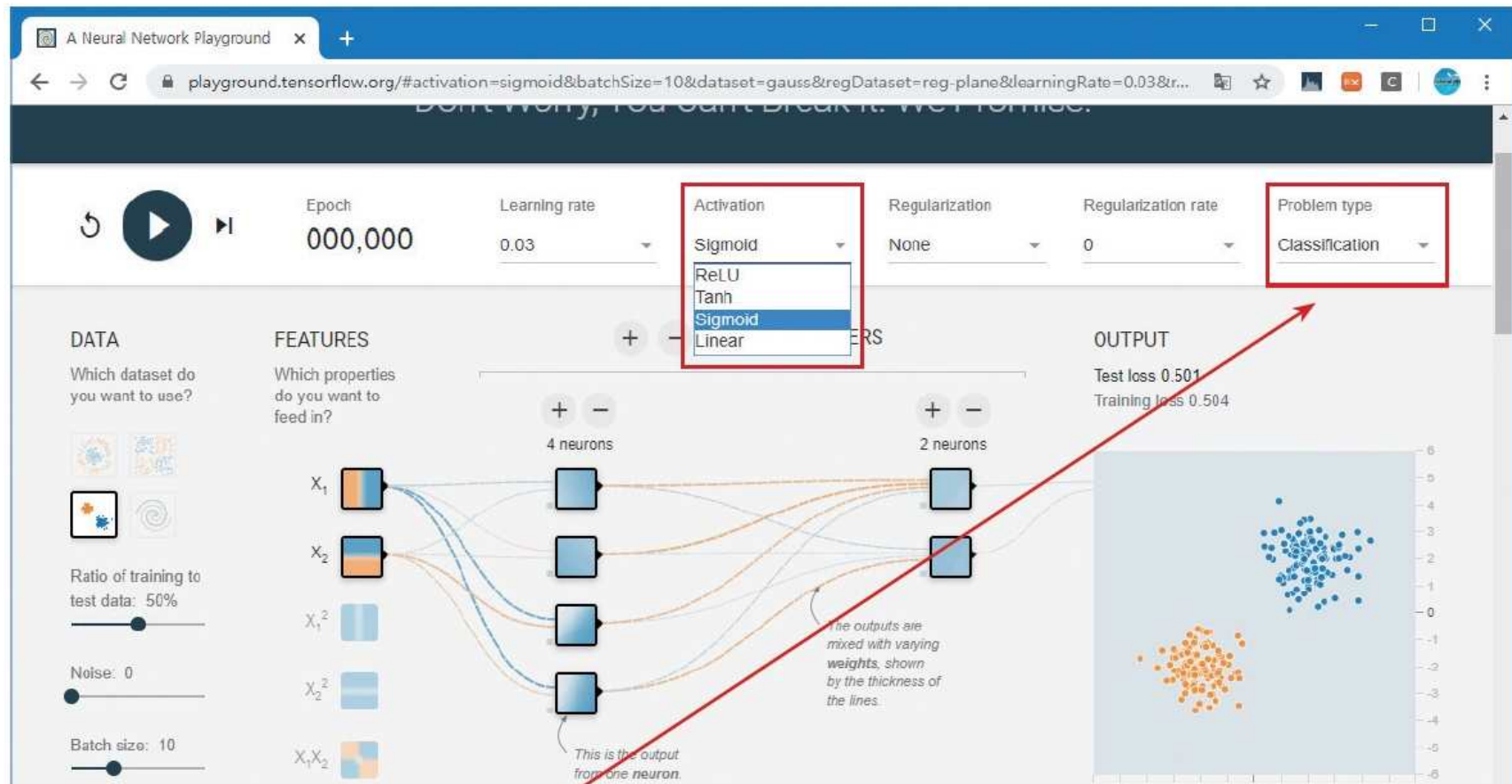
Epoch



학습률

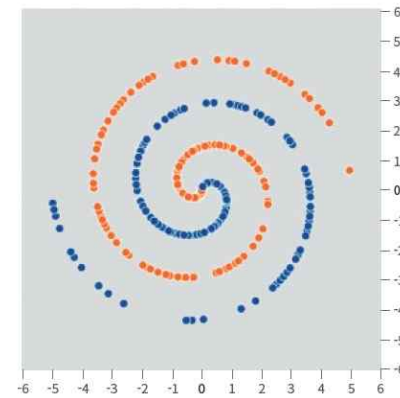
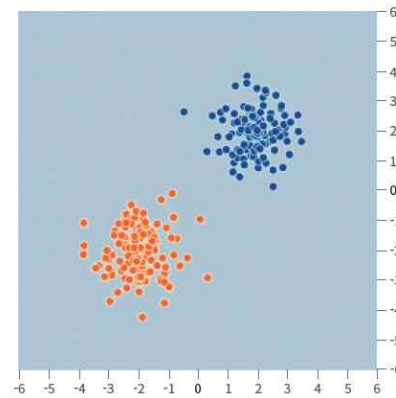
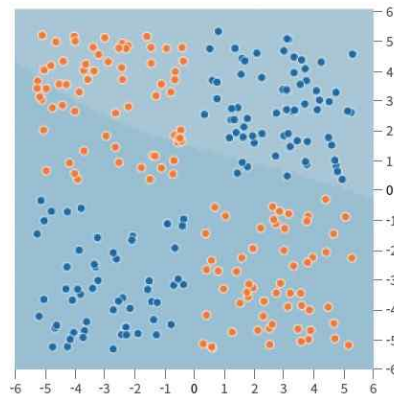
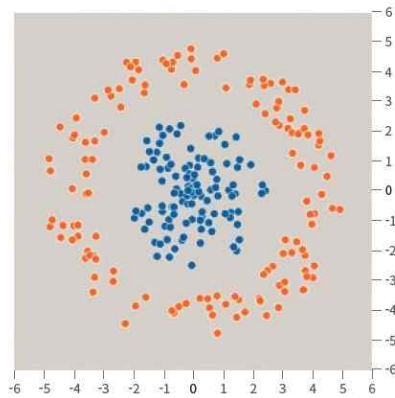


활성화 함수 선택

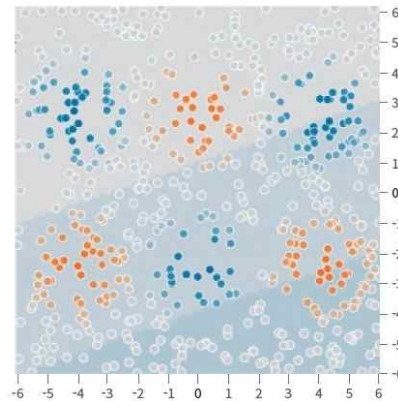
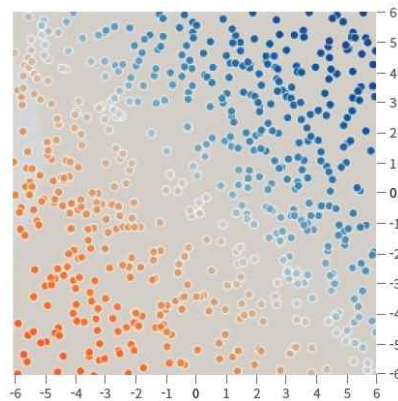


문제 유형

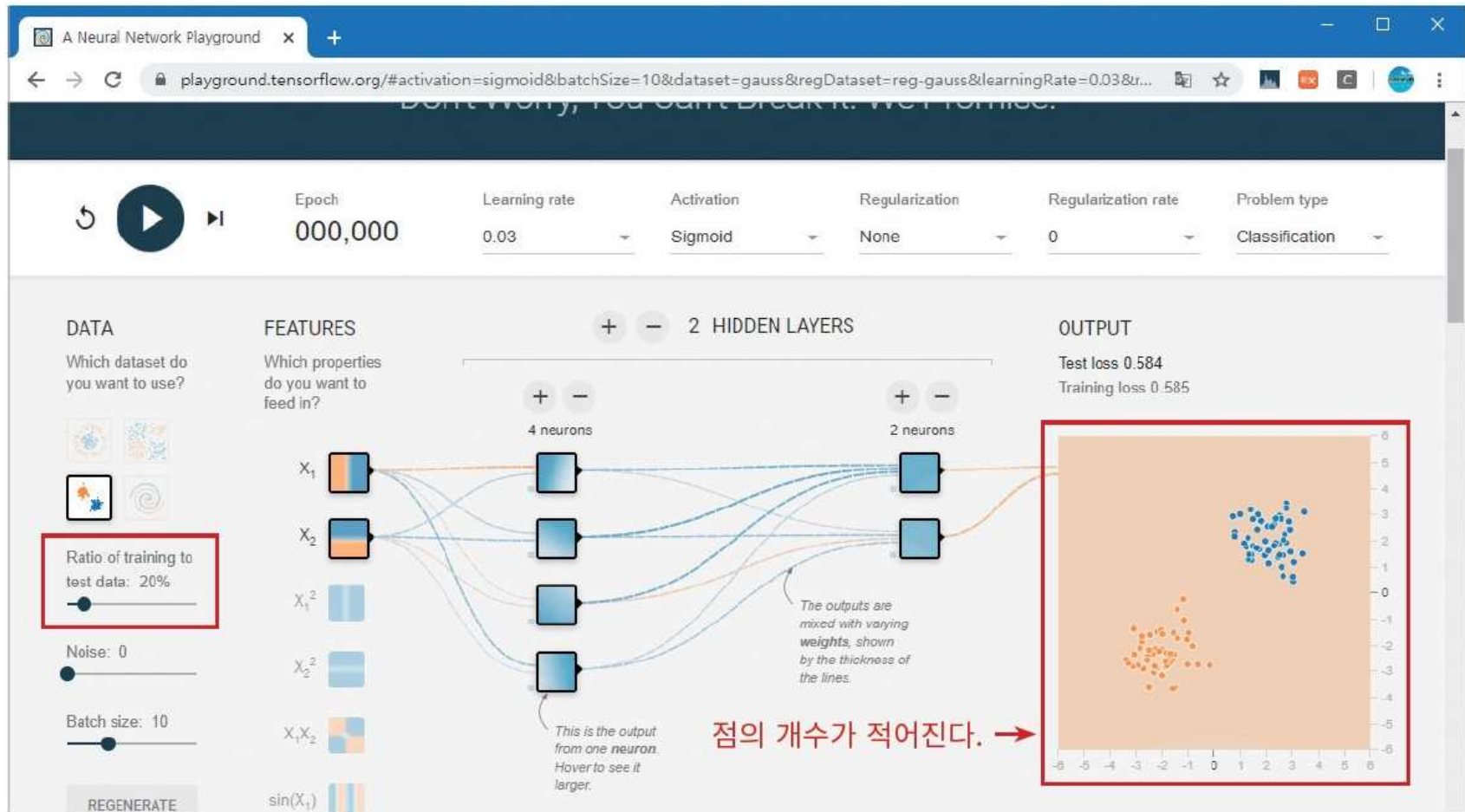
분류 문제



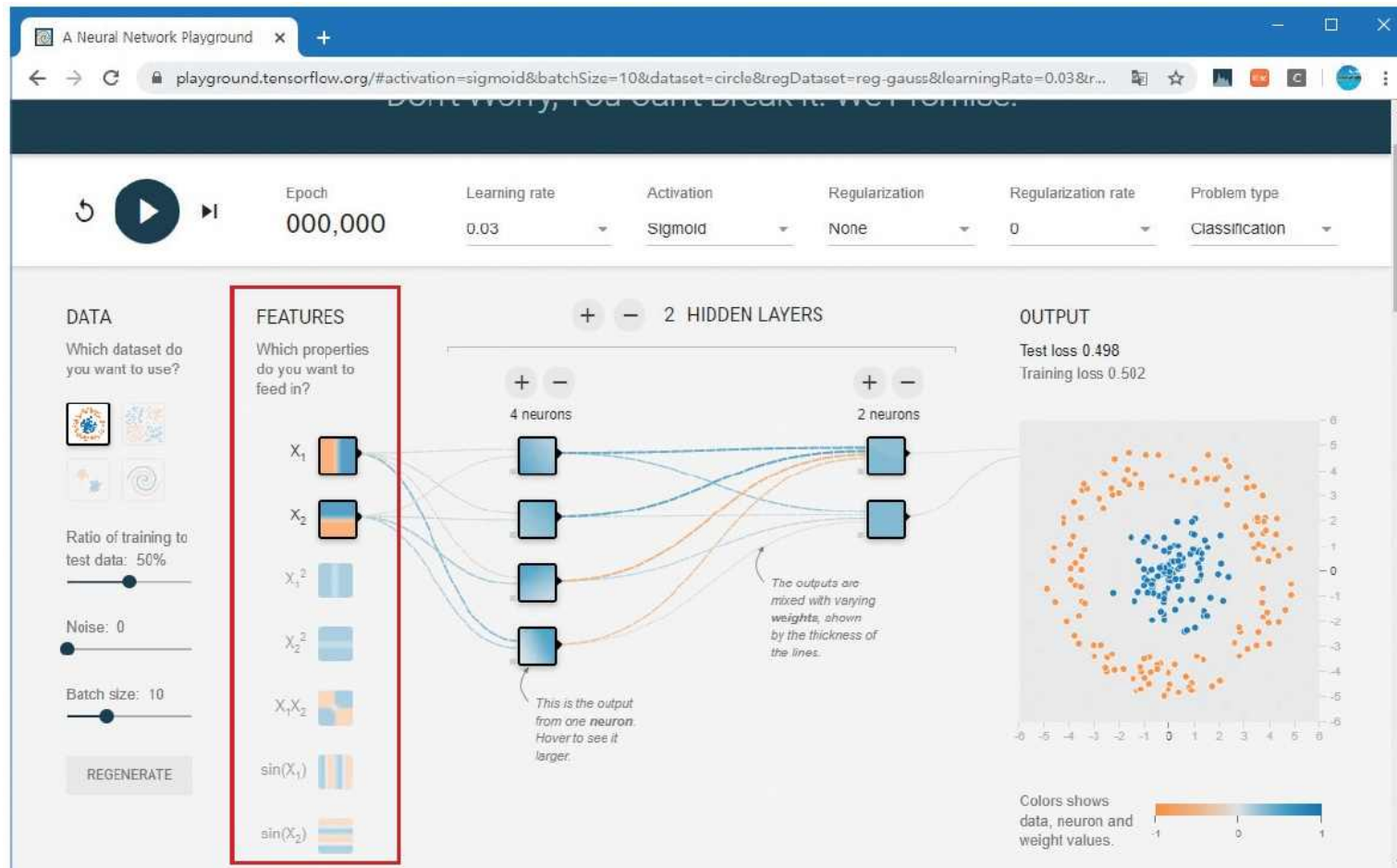
회귀 문제



학습 데이터와 테스트 데이터의 비율



입력 특징 선택



입력 특징

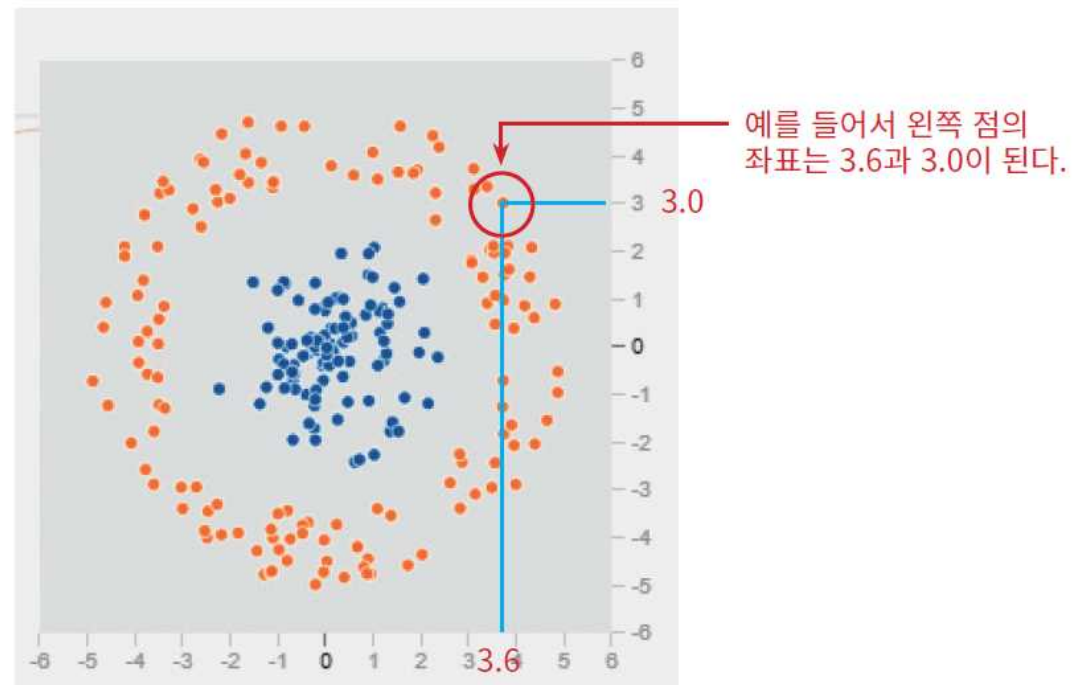


그림 13-29 입력은 점의 좌표이다.

은닉층 추가하기



학습 시작

학습률을 선택할 수 있다.

하나의 에포크는 훈련 데이터를 한번 학습하는 것이다.

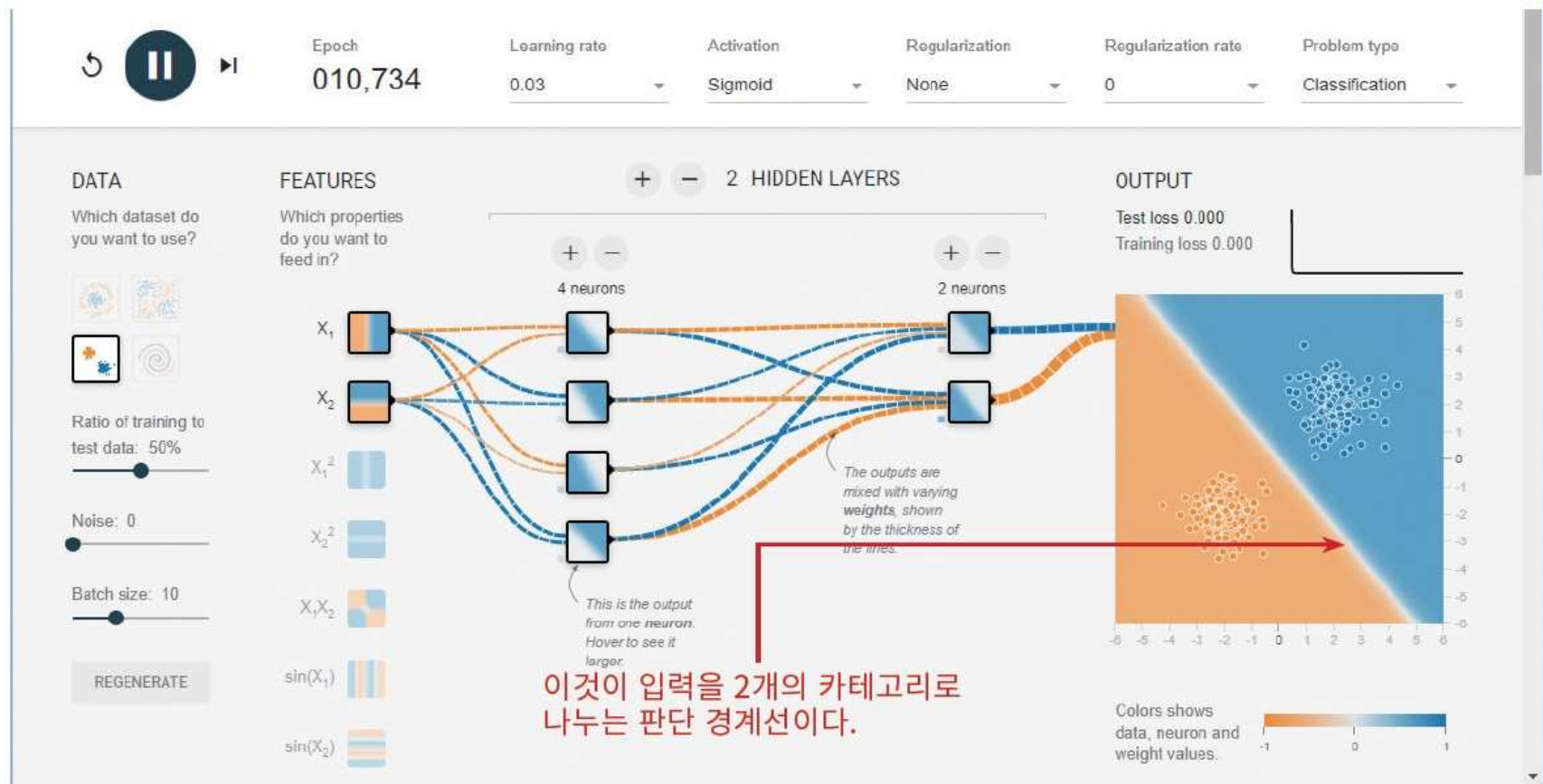
활성화 함수를 선택할 수 있다.

학습이 진행되면서 손실 함수의 값을 볼 수 있다.

The screenshot shows the TensorFlow Playground interface. At the top, there are controls for Epoch (000,193), Learning rate (0.03), Activation (Sigmoid), Regularization (None), Regularization rate (0), and Problem type (Classification). Below these are sliders for DATA (Ratio of training to test data: 50%, Noise: 0, Batch size: 10) and a REGENERATE button. The FEATURES section shows input features X_1 , X_2 , X_1^2 , X_2^2 , X_1X_2 , $\sin(X_1)$, and $\sin(X_2)$. The network structure is 2 HIDDEN LAYERS: 4 neurons in the first hidden layer and 2 neurons in the second. A tooltip indicates a weight of -1.7. The OUTPUT section shows Test loss 0.002 and Training loss 0.002, along with a scatter plot of data points and a color-coded decision boundary. A legend at the bottom right explains that colors show data, neuron, and weight values.

마우스 커서를 이용하면 연결선의 가중치를 알 수 있다.

학습 완료



은닉층 없이 분류 실습

A Neural Network Playground

시그모이드 함수를 선택한다.

playground.tensorflow.org/#activation=sigmoid&batchSize=10&dataset=xor®Dataset=reg-gauss&learningRate=0.03®...

Don't Worry, You Can't Break It. We Promise.

Epoch 006,037 Learning rate 0.03 Activation Sigmoid Regularization None Regularization rate 0 Problem type Classification

XOR와 유사한 데이터를 선택한다.

DATA Which dataset do you want to use?

FEATURES Which properties do you want to feed in?

1 HIDDEN LAYER

2 neurons

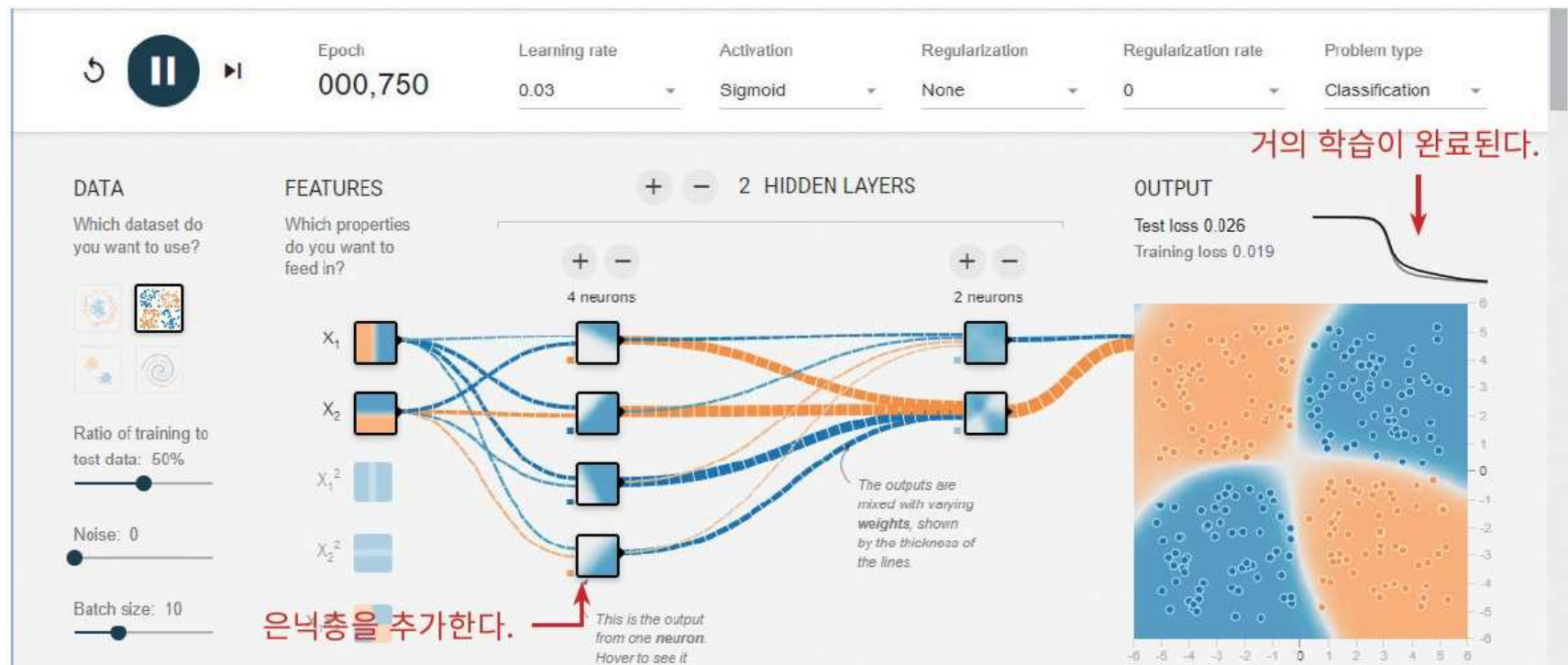
OUTPUT Test loss 0.277 Training loss 0.180

학습이 완료되지 않는다.

출력 노드를 두 개만 남긴다. 은닉층은 없다.

This is the output from one neuron. Hover to see it larger.

은닉층을 추가한 실습



08 구글의 텐서플로우

- 텐서플로우(TensorFlow)는 딥러닝 프레임워크의 일종이다. 텐서플로우는 내부적으로 C/C++로 구현되어 있고 파이썬을 비롯하여 여러가지 언어에서 접근할 수 있도록 인터페이스를 제공한다.

1
2
3
4
5
6

1차원 텐서

3	1	4	1
5	9	2	6
5	3	5	8
9	7	5	3
2	3	8	4
6	2	6	4

2차원 텐서

2	7	8	8	1	8
2	8	4	5	0	5
2	3	5	3	0	8
7	4	7	1	3	5

3차원 텐서

그림 13-17 텐서의 정의

구글의 텐서플로우

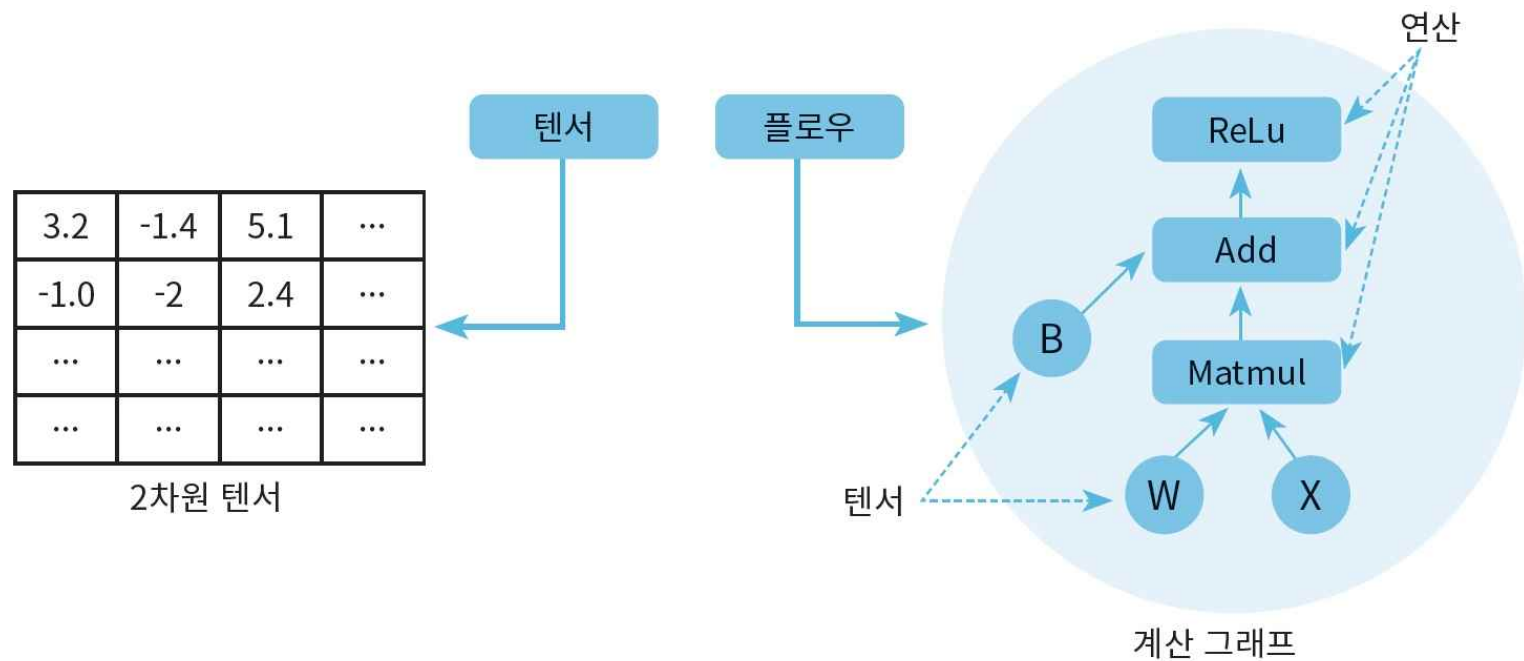
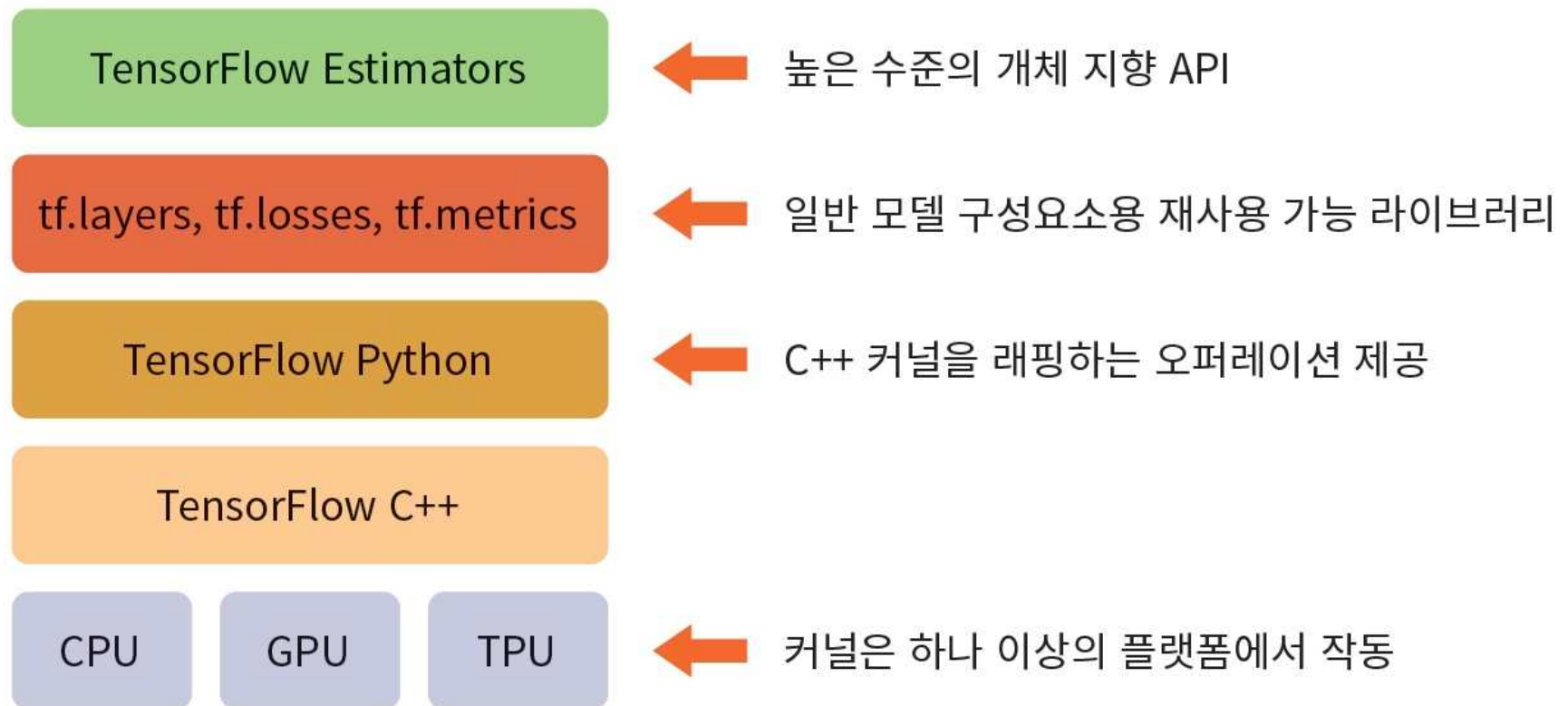


그림 13-18 텐서플로우의 개념

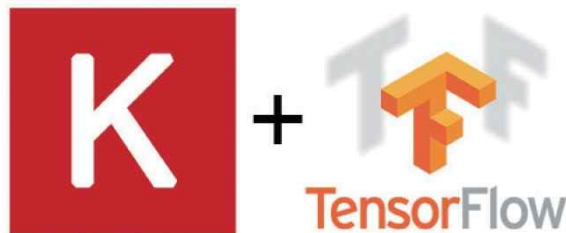
구글의 텐서플로우



(그림 출처: 구글)

Keras

- Keras는 Python으로 작성되었으며 TensorFlow , CNTK 또는 Theano에서 실행할 수 있는 고수준 딥러닝 API이다.
 - 쉽고 빠른 프로토타이핑이 가능하다.
 - 순방향 신경망, 컨볼루션 신경망과 반복적인 신경망은 물론 여러 가지의 조합도 지원한다.
 - CPU 및 GPU에서 원활하게 실행된다.



Keras

- Keras는 신경망을 레고 조립하듯이 만들 수 있다.

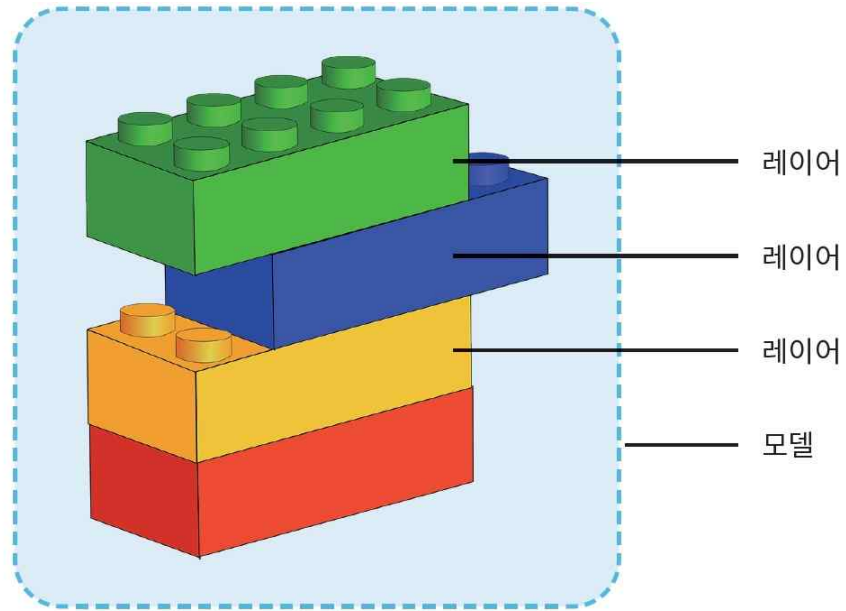


그림 13-19 Keras의 기본 개념

모델 작성

```
from tf.keras.models import Sequential  
  
model = Sequential()  
  
from tf.keras.layers import Dense  
  
model.add(Dense(units=64, activation='sigmoid', input_dim=100))    # ①  
model.add(Dense(units=10, activation='sigmoid'))                  # ②
```

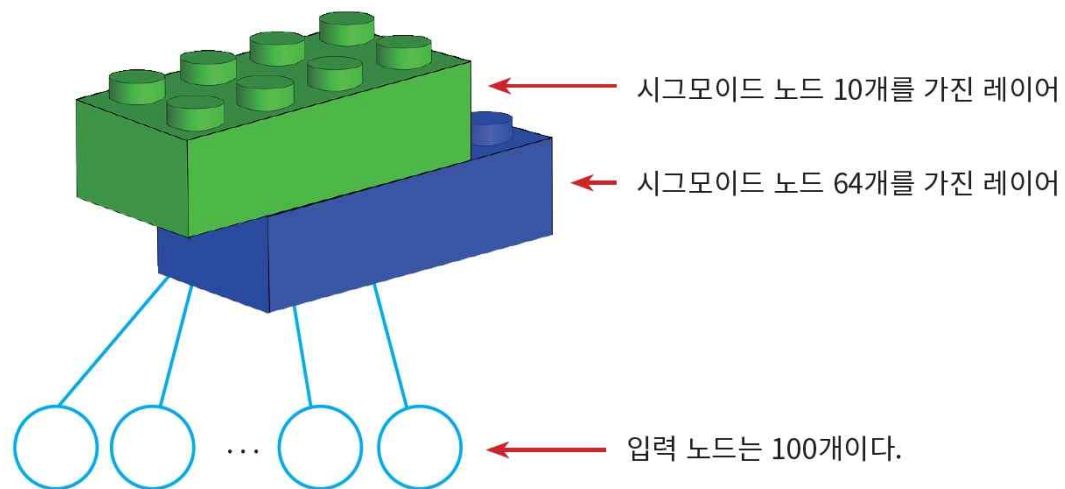


그림 13-20 Keras에서의 레이어

학습 과정 정의

```
model.compile(loss='mse',  
              optimizer='sgd',  
              metrics=['accuracy'])
```

```
model.fit(X, y, epochs=5, batch_size=32)
```

학습

```
loss_and_metrics = model.evaluate(X, y, batch_size=128)
```

```
classes = model.predict(new_X, batch_size=128)
```

평가와 예측

Keras 예제 #1 선형 회귀

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

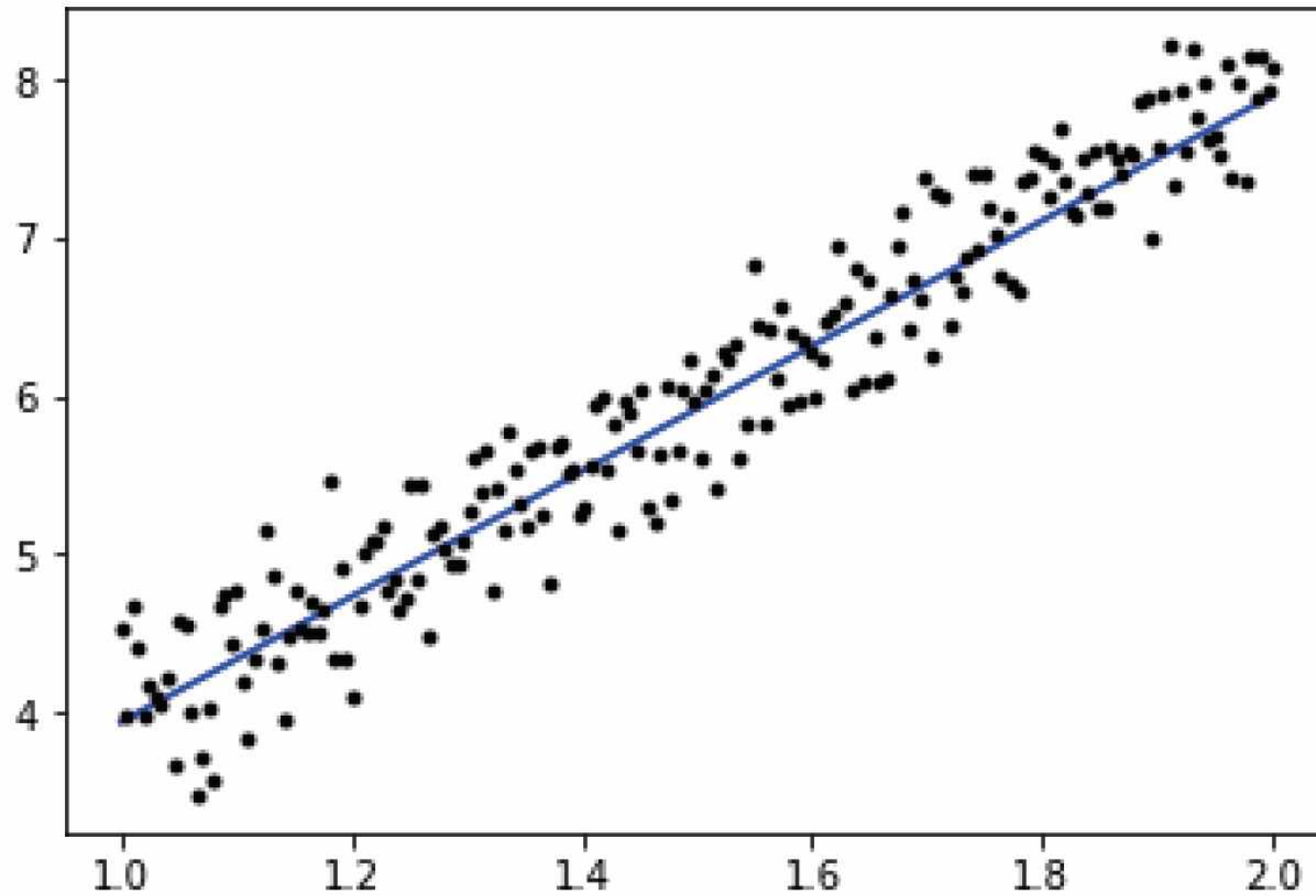
# 가상의 데이터 생성
X = data = np.linspace(1,2,200)      # 시작값=1, 종료값=2, 개수=200
y = x*4 + np.random.randn(200) * 0.3 # x를 4배로 하고 편차 0.3정도의 가우시안
                                     # 잡음추가

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Dense(1, input_dim=1, activation='linear'))
model.compile(optimizer='sgd', loss='mse', metrics=['mse'])
model.fit(X, y, batch_size=1, epochs=30)

predict = model.predict(data)

plt.plot(data, predict, 'b', data, y, 'k.') # 첫 번째 그래프는 파란색 마커로
plt.show()                                # 두 번째 그래프는 검정색 .으로 그린다.
```

Keras 예제 #1 실행결과



Keras 예제 #2 XOR

```
import tensorflow as tf
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Dense(2, input_dim=2, activation='sigmoid'))
model.add(tf.keras.layers.Dense(1, activation='sigmoid'))

sgd = tf.keras.optimizers.SGD(lr=0.1)
model.compile(loss='mean_squared_error', optimizer=sgd)

model.fit(X, y, batch_size=1, epochs=1000)
print(model.predict(X))
```

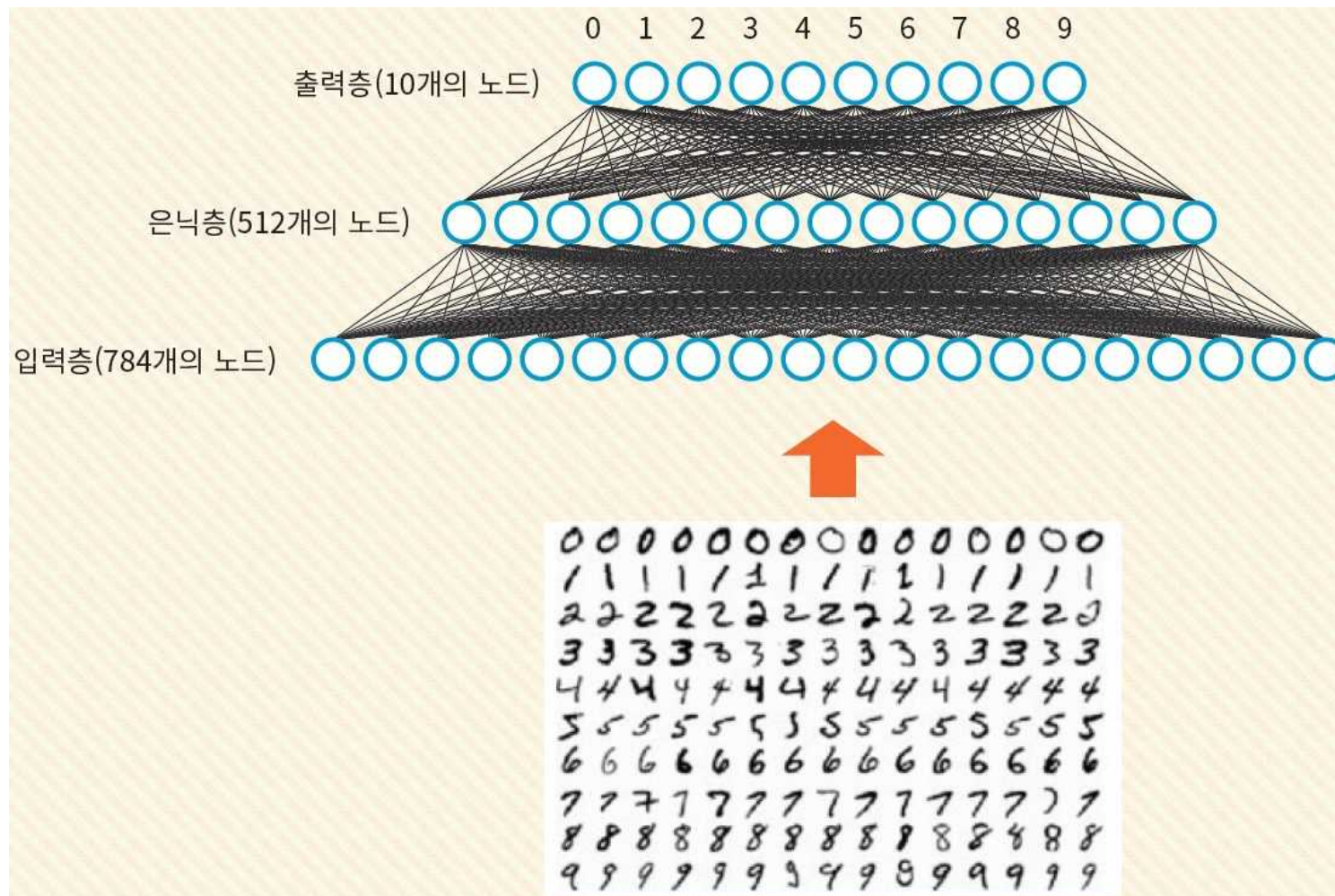
Keras 예제 #2 실행결과

```
[[0.19608304]  
 [0.64313614]  
 [0.6822454 ]  
 [0.53627  ]]
```

```
[[0.02743807]  
 [0.9702845 ]  
 [0.9704155 ]  
 [0.03712982]]
```

반복횟수(에포크)를
10000으로 늘린다면

Lab: MLP를 사용한 MNIST 숫자인식



Lab: MLP를 사용한 MNIST 숫자인식

```
import tensorflow as tf

batch_size = 128 # 가중치를 변경하기 전에 처리하는 샘플의 개수
num_classes = 10 # 출력 클래스의 개수
epochs = 20      # 에포크의 개수

# 데이터를 학습 데이터와 테스트 데이터로 나눈다.
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# 입력 이미지를 2차원에서 1차원 벡터로 변경한다.
x_train = x_train.reshape(60000, 784)
x_test = x_test.reshape(10000, 784)

# 입력 이미지의 픽셀 값이 0.0에서 1.0 사이의 값이 되게 한다.
x_train = x_train.astype('float32')
x_test = x_test.astype('float32')
x_train /= 255
x_test /= 255
```

Lab: MLP를 사용한 MNIST 숫자인식

```
# 클래스의 개수에 따라서 하나의 출력 픽셀만이 1이 되게 한다.  
# 예를 들면 1 0 0 0 0 0 0 0 0과 같다.
```

```
y_train = tf.keras.utils.to_categorical(y_train, num_classes)  
y_test = tf.keras.utils.to_categorical(y_test, num_classes)
```

```
# 신경망의 모델을 구축한다.  
model = tf.keras.models.Sequential()  
model.add(tf.keras.layers.Dense(512, activation='sigmoid', input_shape=(784,)))  
model.add(tf.keras.layers.Dense(num_classes, activation='sigmoid'))
```

```
model.summary()
```

```
sgd = tf.keras.optimizers.SGD(lr=0.1)
```

Lab: MLP를 사용한 MNIST 숫자인식

```
# 손실 함수를 제공 오차 함수로 설정하고 학습 알고리즘은 SGD 방식으로 한다.
model.compile(loss='mean_squared_error',
              optimizer=sgd,
              metrics=['accuracy'])

# 학습을 수행한다.
history = model.fit(x_train, y_train,
                   batch_size=batch_size,
                   epochs=epochs)

# 학습을 평가한다.
score = model.evaluate(x_test, y_test, verbose=0)
print('테스트 손실값:', score[0])
print('테스트 정확도:', score[1])
```

실행결과

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 512)	401920
dense_5 (Dense)	(None, 10)	5130

=====
Total params: 407,050
Trainable params: 407,050
Non-trainable params: 0
...
Epoch 18/20
60000/60000 [=====] - 1s 18us/sample - loss: 0.0344 -
acc: 0.8480
Epoch 19/20
60000/60000 [=====] - 1s 18us/sample - loss: 0.0335 -
acc: 0.8505
Epoch 20/20
60000/60000 [=====] - 1s 18us/sample - loss: 0.0328 -
acc: 0.8530
테스트 손실값: 0.03148304631710053
테스트 정확도: 0.8628

Summary

- 입력층과 출력층 사이에 은닉층(hidden layer)을 가지고 있는 신경망을 다층 퍼셉트론(multilayer perceptron: MLP)이라고 부른다.
- MLP를 학습시키기 위하여 역전파 알고리즘(back-propagation)이 재발견되었다. 이 알고리즘이 지금까지도 신경망 학습 알고리즘의 근간이 되고 있다.
- 역전파 알고리즘은 입력이 주어지면 순방향으로 계산하여 출력을 계산한 후에 실제 출력과 우리가 원하는 출력 간의 오차를 계산한다. 이 오차를 역방향으로 전파하면서 오차를 줄이는 방향으로 가중치를 변경한다.

Q & A

